

Large Foundation Models: Mathematics, Algorithms, and Applications

Pipi Hu

November 18, 2025

Contents

Preface

xiii

I Generative Models 1

1 Diffusion Models: From Mathematical Foundations to Practical Implementation	3
1.1 Introduction	3
1.2 Preliminary: Generating Samples from Probability Distributions	3
1.2.1 The Challenge of Sampling	3
1.2.2 The Role of the Score Function	4
1.2.3 Continuous-Time Formulation	4
1.3 Matching Score for Training	5
1.3.1 The Challenge of Score Matching	5
1.3.2 Implicit Score Matching	5
1.3.3 Denoising Score Matching	5
1.4 Relations Between Different Models	6
1.4.1 Gaussian Noise Addition	6
1.4.2 Tweedie’s Formula	6
1.4.3 Alternative Derivation from Score Definition	6
1.4.4 Theoretical Foundation: Conditional Expectation	7
1.4.5 Three Equivalent Model Formulations	7
1.5 Potential Score Matching Loss	7
1.5.1 Key Identity: Jacobian Switch	7
1.5.2 Score as Conditional Force Expectation	8
1.6 From Theory to Practice: Implementation Approaches	8
1.6.1 Score Matching Langevin Dynamics (SMLD)	9
1.6.2 Denoising Diffusion Probabilistic Models (DDPM)	9
1.7 Continuous Formulations	9
1.7.1 Continuous Version of SMLD	9
1.7.2 Continuous Version of DDPM	10
1.7.3 Variable Elimination (VE) and Variable Preservation (VP)	10
1.7.4 Transition Kernels for VE and VP	10
1.8 Training and Inference	11
1.8.1 Training Recipe	11
1.8.2 Inference: Reverse Denoising Process	11
1.8.3 Inference Recipe	11
1.8.4 The Corrector: Langevin MCMC	11
1.9 Recent Advances and Practical Considerations	12
1.9.1 DDPM: Denoising Diffusion Probabilistic Models	12
1.9.2 EDM: Elucidating the Design Space	13
1.9.3 DPM-Solver: Fast ODE Solver	13

1.9.4	Consistency Models	13
1.10	Open Questions and Future Directions	14
1.11	Conclusion	14
2	Flow Matching: A Unifying Perspective on Generative Modeling	15
2.1	Introduction	15
2.2	Mathematical Foundations	15
2.2.1	The Continuity Equation	15
2.2.2	Conditional Probability Paths	15
2.3	Gaussian Conditional Paths	16
2.3.1	Mathematical Derivation	16
2.3.2	Path Choices	16
2.4	Training and Inference	16
2.4.1	Flow Matching Objective	16
2.4.2	Mathematical Optimality	17
2.4.3	Inference	17
2.5	Connection to Diffusion Models	17
2.5.1	Probability Flow ODE	17
2.6	MeanFlow: Average Velocity Field	17
2.6.1	Motivation	17
2.6.2	Definition	17
2.6.3	MeanFlow Identity	18
2.6.4	Training and Sampling	18
2.7	Recent Advances and Applications	18
2.7.1	Practical Considerations	18
2.7.2	Applications	18
2.8	Conclusion	18
3	Variational Autoencoders: Theory and Applications	21
3.1	Introduction	21
3.2	The Challenge of Generative Modeling	21
3.2.1	Problem Formulation	21
3.2.2	Traditional Approaches and Limitations	21
3.3	Variational Inference Framework	22
3.3.1	The ELBO	22
3.3.2	ELBO Interpretation	22
3.4	VAE Architecture and Training	22
3.4.1	Architecture	22
3.4.2	The Reparameterization Trick	22
3.4.3	Training Objective	23
3.4.4	Analytical KL Divergence	23
3.5	VAE Variants and Extensions	23
3.5.1	Beta-VAE: Controlling Regularization	23
3.5.2	VQ-VAE: Vector Quantized VAE	23
3.5.3	WAE: Wasserstein Autoencoder	24
3.5.4	VAE-GAN: Combining VAE and GAN	24
3.6	Advanced Topics	24
3.6.1	Hierarchical VAE	24
3.6.2	Conditional VAE (C-VAE)	24
3.6.3	Importance Weighted Autoencoder (IWAE)	25
3.7	Implementation and Practical Considerations	25
3.7.1	Common Issues and Solutions	25

3.7.2	Training Tricks	25
3.8	Recent Advances and Applications	25
3.8.1	Transformer-based VAEs	25
3.8.2	Improved Training	26
3.8.3	Applications	26
3.9	Evaluation Metrics	26
3.10	Open Questions and Future Directions	26
3.10.1	Theoretical Understanding	26
3.10.2	Technical Challenges	26
3.10.3	Applications	27
3.11	Conclusion	27
4	Normalizing Flows: Exact Likelihood Generative Modeling	29
4.1	Introduction	29
4.2	Mathematical Foundations	29
4.2.1	Change of Variables Theorem	29
4.2.2	Proof of Change of Variables Formula	29
4.2.3	Normalizing Flows	30
4.3	Autoregressive Flows	30
4.3.1	Autoregressive Transformations	30
4.3.2	Masked Autoregressive Flow (MAF)	30
4.3.3	Inverse Autoregressive Flow (IAF)	31
4.3.4	TarFlow: Transformer-based Autoregressive Flow	31
4.4	Coupling Layers	32
4.4.1	Affine Coupling Layers	32
4.4.2	Real NVP: Real-valued Non-volume Preserving	32
4.5	Continuous Normalizing Flows	32
4.5.1	Neural Ordinary Differential Equations	32
4.5.2	Proof of Neural ODE Change of Variables Formula	33
4.5.3	FFJORD: Free-form Jacobian of Reversible Dynamics	33
4.6	Conditional Normalizing Flows	33
4.6.1	Goal and Conditional Likelihood	33
4.6.2	Conditioning and Architecture	34
4.7	Training and Applications	34
4.7.1	Maximum Likelihood Training	34
4.7.2	Applications	34
4.8	Comparison with Other Methods	35
4.8.1	Normalizing Flows vs. Other Generative Models	35
4.9	Challenges and Future Directions	35
4.9.1	Current Challenges	35
4.9.2	Future Directions	35
4.10	Conclusion	36
5	Generative Adversarial Networks	37
5.1	Introduction	37
5.2	Mathematical Foundations	37
5.2.1	The Minimax Game	37
5.2.2	Optimal Discriminator	37
5.2.3	Global Optimum	38
5.3	Training Dynamics and Challenges	38
5.3.1	Training Algorithm	38
5.3.2	Mode Collapse	38

5.3.3	Training Instability	39
5.4	Improved GAN Architectures	39
5.4.1	DCGAN: Deep Convolutional GANs	39
5.4.2	WGAN: Wasserstein GAN	39
5.4.3	WGAN-GP: Gradient Penalty	39
5.5	Loss Functions and Training Techniques	40
5.5.1	Alternative Loss Functions	40
5.5.2	Training Techniques	40
5.6	Evaluation Metrics	40
5.6.1	Inception Score (IS)	40
5.6.2	Fréchet Inception Distance (FID)	40
5.6.3	Precision and Recall	40
5.7	Recent Advances	41
5.7.1	StyleGAN: Style-Based Generator	41
5.7.2	BigGAN: Large Scale GAN Training	41
5.7.3	Self-Attention GAN (SAGAN)	41
5.8	Applications	41
5.8.1	Image Generation	41
5.8.2	Data Augmentation	42
5.8.3	Scientific Applications	42
5.9	Challenges and Limitations	42
5.9.1	Training Challenges	42
5.9.2	Theoretical Limitations	42
5.10	Future Directions	42
5.10.1	Theoretical Advances	42
5.10.2	Architectural Improvements	42
5.10.3	Applications	43
5.11	Conclusion	43
II	Autoregressive Models	45
6	Autoregressive Models: Sequential Generation and Language Modeling	47
6.1	Introduction	47
6.2	Mathematical Foundations	47
6.2.1	Autoregressive Factorization	47
6.2.2	Maximum Likelihood Training	47
6.2.3	Sampling	48
6.3	Neural Autoregressive Models	48
6.3.1	Neural Language Models	48
6.3.2	Recurrent Neural Networks (RNNs)	48
6.3.3	Long Short-Term Memory (LSTM)	48
6.4	Transformer Architecture	49
6.4.1	Self-Attention Mechanism	49
6.4.2	Multi-Head Attention	49
6.4.3	Positional Encoding	49
6.4.4	Transformer Block	49
6.5	GPT: Generative Pre-trained Transformer	49
6.5.1	Architecture	49
6.5.2	Training	49
6.5.3	Pre-training and Fine-tuning	50
6.6	BERT: Bidirectional Encoder Representations	50

6.6.1	Masked Language Modeling	50
6.6.2	Next Sentence Prediction	50
6.7	Advanced Architectures	50
6.7.1	T5: Text-to-Text Transfer Transformer	50
6.7.2	PaLM: Pathways Language Model	50
6.8	Training Techniques	50
6.8.1	Gradient Accumulation	50
6.8.2	Mixed Precision Training	51
6.8.3	Gradient Clipping	51
6.9	Evaluation Metrics	51
6.9.1	Perplexity	51
6.9.2	BLEU Score	51
6.9.3	ROUGE Score	51
6.10	Applications	51
6.10.1	Language Modeling	51
6.10.2	Machine Translation	52
6.10.3	Text Summarization	52
6.10.4	Dialog Systems	52
6.11	Challenges and Limitations	52
6.11.1	Computational Challenges	52
6.11.2	Theoretical Limitations	52
6.11.3	Practical Challenges	52
6.12	Recent Advances	53
6.12.1	Efficient Attention Mechanisms	53
6.12.2	Long-Range Dependencies	53
6.13	Future Directions	53
6.13.1	Architectural Improvements	53
6.13.2	Training Improvements	53
6.13.3	Applications	54
6.14	Conclusion	54

III Advanced Topics 55

7	Discrete Diffusion Models: Extending Diffusion to Discrete Data	57
7.1	Introduction	57
7.2	Mathematical Foundations	57
7.2.1	Discrete State Spaces	57
7.2.2	Forward Process	57
7.2.3	Reverse Process	57
7.3	Noise Schedules for Discrete Data	58
7.3.1	Uniform Noise	58
7.3.2	Gaussian-like Noise	58
7.4	Training Objectives	58
7.4.1	Denoising Score Matching	58
7.4.2	Simplified Objective	58
7.5	Applications to Text	58
7.5.1	Text Diffusion	58
7.5.2	Training Procedure	58
7.5.3	Generation Procedure	59
7.6	Advanced Techniques	59
7.6.1	Learned Noise Schedules	59

7.6.2	Conditional Generation	59
7.6.3	Classifier-Free Guidance	59
7.7	Discrete Diffusion for Images	59
7.7.1	VQ-VAE Integration	59
7.7.2	Benefits	59
7.8	Evaluation Metrics	60
7.8.1	Perplexity	60
7.8.2	BLEU Score	60
7.8.3	FID Score	60
7.9	Challenges and Limitations	60
7.9.1	Computational Challenges	60
7.9.2	Modeling Challenges	60
7.10	Recent Advances	60
7.10.1	Improved Noise Schedules	60
7.10.2	Better Training Techniques	60
7.11	Applications	61
7.11.1	Text Generation	61
7.11.2	Image Generation	61
7.11.3	Scientific Applications	61
7.12	Future Directions	61
7.12.1	Theoretical Advances	61
7.12.2	Architectural Improvements	61
7.12.3	Applications	61
7.13	Conclusion	62
8	World Models: Learning Internal Representations of Environments	63
8.1	Introduction	63
8.2	Mathematical Foundations	63
8.2.1	World Model Definition	63
8.2.2	Components of World Models	63
8.3	Training Objectives	64
8.3.1	Reconstruction Loss	64
8.3.2	Prediction Loss	64
8.3.3	Reward Prediction	64
8.4	Architecture Designs	64
8.4.1	Recurrent World Models	64
8.4.2	Transformer-based World Models	64
8.4.3	VAE-based World Models	64
8.5	Planning with World Models	64
8.5.1	Model Predictive Control (MPC)	64
8.5.2	Tree Search	65
8.6	Applications	65
8.6.1	Atari Games	65
8.6.2	Robotics	65
8.6.3	Autonomous Driving	65
8.7	Challenges and Limitations	65
8.7.1	Model Accuracy	65
8.7.2	Computational Challenges	65
8.8	Recent Advances	66
8.8.1	Improved Architectures	66
8.8.2	Better Training Techniques	66

8.9	Future Directions	66
8.9.1	Theoretical Advances	66
8.9.2	Architectural Improvements	66
8.9.3	Applications	66
8.10	Conclusion	66
9	Reinforcement Learning: Learning to Act in Dynamic Environments	69
9.1	Introduction	69
9.2	Mathematical Foundations	69
9.2.1	Markov Decision Process (MDP)	69
9.2.2	Policy and Value Functions	69
9.2.3	Bellman Equations	70
9.3	Value-Based Methods	70
9.3.1	Value Iteration	70
9.3.2	Policy Iteration	70
9.3.3	Q-Learning	70
9.3.4	Deep Q-Networks (DQN)	70
9.4	Policy-Based Methods	70
9.4.1	Policy Gradient	70
9.4.2	REINFORCE	71
9.4.3	Actor-Critic Methods	71
9.5	Advanced Algorithms	71
9.5.1	Proximal Policy Optimization (PPO)	71
9.5.2	Trust Region Policy Optimization (TRPO)	71
9.5.3	Soft Actor-Critic (SAC)	71
9.6	Function Approximation	71
9.6.1	Neural Networks	71
9.6.2	Experience Replay	72
9.6.3	Target Networks	72
9.7	Exploration Strategies	72
9.7.1	Epsilon-Greedy	72
9.7.2	Upper Confidence Bound (UCB)	72
9.7.3	Thompson Sampling	72
9.8	Applications	72
9.8.1	Games	72
9.8.2	Robotics	73
9.8.3	Autonomous Systems	73
9.9	Challenges and Limitations	73
9.9.1	Sample Efficiency	73
9.9.2	Stability	73
9.9.3	Safety	73
9.10	Recent Advances	73
9.10.1	Model-Based RL	73
9.10.2	Multi-Agent RL	74
9.10.3	Hierarchical RL	74
9.11	Future Directions	74
9.11.1	Theoretical Advances	74
9.11.2	Algorithmic Improvements	74
9.11.3	Applications	74
9.12	Conclusion	74
10	Reasoning: From Symbolic Logic to Neural Reasoning	77

10.1	Introduction	77
10.2	Symbolic Reasoning	77
10.2.1	Propositional Logic	77
10.2.2	Predicate Logic	77
10.2.3	Inference Rules	77
10.3	Neural Reasoning	78
10.3.1	Neural Symbolic Reasoning	78
10.3.2	Graph Neural Networks for Reasoning	78
10.3.3	Attention Mechanisms for Reasoning	78
10.4	Mathematical Reasoning	78
10.4.1	Arithmetic Reasoning	78
10.4.2	Algebraic Reasoning	78
10.4.3	Geometric Reasoning	78
10.5	Logical Reasoning	79
10.5.1	Propositional Reasoning	79
10.5.2	Predicate Reasoning	79
10.6	Common Sense Reasoning	79
10.6.1	Physical Reasoning	79
10.6.2	Causal Reasoning	79
10.7	Reasoning in Language	79
10.7.1	Natural Language Inference	79
10.7.2	Question Answering	80
10.8	Reasoning in Vision	80
10.8.1	Spatial Reasoning	80
10.8.2	Temporal Reasoning	80
10.9	Reasoning Architectures	80
10.9.1	Memory Networks	80
10.9.2	Neural Turing Machines	80
10.9.3	Transformer-based Reasoning	81
10.10	Evaluation Metrics	81
10.10.1	Accuracy	81
10.10.2	Reasoning-specific Metrics	81
10.11	Challenges and Limitations	81
10.11.1	Symbolic vs. Neural	81
10.11.2	Generalization	81
10.11.3	Interpretability	81
10.12	Recent Advances	82
10.12.1	Large Language Models	82
10.12.2	Chain-of-Thought Reasoning	82
10.12.3	Program Synthesis	82
10.13	Applications	82
10.13.1	Scientific Discovery	82
10.13.2	Legal Reasoning	82
10.13.3	Medical Diagnosis	82
10.14	Future Directions	83
10.14.1	Theoretical Advances	83
10.14.2	Architectural Improvements	83
10.14.3	Applications	83
10.15	Conclusion	83

11.1	Introduction	85
11.2	Agent Architecture	85
11.2.1	Basic Agent Model	85
11.2.2	Agent-Environment Interaction	85
11.3	Types of Agents	85
11.3.1	Reactive Agents	85
11.3.2	Deliberative Agents	86
11.3.3	Hybrid Agents	86
11.4	Learning in Agents	86
11.4.1	Reinforcement Learning	86
11.4.2	Imitation Learning	86
11.4.3	Transfer Learning	86
11.5	Multi-Agent Systems	86
11.5.1	Cooperative Agents	86
11.5.2	Competitive Agents	87
11.5.3	Mixed Environments	87
11.6	Agent Communication	87
11.6.1	Message Passing	87
11.6.2	Shared Memory	87
11.6.3	Protocols	87
11.7	Agent Planning	88
11.7.1	Classical Planning	88
11.7.2	Probabilistic Planning	88
11.7.3	Hierarchical Planning	88
11.8	Agent Learning	88
11.8.1	Online Learning	88
11.8.2	Offline Learning	88
11.8.3	Meta-Learning	88
11.9	Agent Architectures	89
11.9.1	Belief-Desire-Intention (BDI)	89
11.9.2	Cognitive Architectures	89
11.9.3	Neural Architectures	89
11.10	Agent Applications	89
11.10.1	Robotics	89
11.10.2	Autonomous Systems	89
11.10.3	Software Agents	89
11.11	Agent Evaluation	90
11.11.1	Performance Metrics	90
11.11.2	Benchmarks	90
11.12	Challenges and Limitations	90
11.12.1	Scalability	90
11.12.2	Safety	90
11.12.3	Ethics	90
11.13	Recent Advances	91
11.13.1	Large Language Models as Agents	91
11.13.2	Multi-Modal Agents	91
11.13.3	Embodied Agents	91
11.14	Future Directions	91
11.14.1	Theoretical Advances	91
11.14.2	Architectural Improvements	91
11.14.3	Applications	91

11.15 Conclusion	92
12 AI for Science: Accelerating Scientific Discovery	93
12.1 Introduction	93
12.2 Mathematical Foundations	93
12.2.1 Scientific Data Representation	93
12.2.2 Physical Constraints	93
12.3 Drug Discovery	94
12.3.1 Molecular Representation	94
12.3.2 Molecular Property Prediction	94
12.3.3 Drug-Target Interaction	94
12.3.4 Molecular Generation	94
12.4 Materials Science	94
12.4.1 Crystal Structure Prediction	94
12.4.2 Property Prediction	94
12.4.3 Materials Discovery	95
12.5 Climate Science	95
12.5.1 Weather Prediction	95
12.5.2 Climate Modeling	95
12.5.3 Extreme Weather Events	95
12.6 Physics	95
12.6.1 Particle Physics	95
12.6.2 Plasma Physics	96
12.6.3 Quantum Physics	96
12.7 Biology	96
12.7.1 Protein Structure Prediction	96
12.7.2 Protein Function Prediction	96
12.7.3 Genomics	96
12.8 Scientific Computing	97
12.8.1 Numerical Methods	97
12.8.2 Surrogate Models	97
12.8.3 Uncertainty Quantification	97
12.9 Scientific Data Analysis	97
12.9.1 Data Preprocessing	97
12.9.2 Feature Engineering	97
12.9.3 Interpretability	98
12.10 Challenges and Limitations	98
12.10.1 Data Challenges	98
12.10.2 Model Challenges	98
12.10.3 Integration Challenges	98
12.11 Recent Advances	98
12.11.1 Large Language Models for Science	98
12.11.2 Multimodal AI	99
12.11.3 Federated Learning	99
12.12 Applications	99
12.12.1 COVID-19 Research	99
12.12.2 Cancer Research	99
12.12.3 Renewable Energy	99
12.13 Future Directions	100
12.13.1 Theoretical Advances	100
12.13.2 Methodological Improvements	100

12.13.3 Applications	100
12.14 Conclusion	100

List of Figures

List of Tables

4.1 Comparison of generative modeling approaches	35
--	----

Preface

This book presents a comprehensive mathematical treatment of large foundation models, covering the theoretical foundations, algorithmic developments, and practical applications that have revolutionized artificial intelligence in recent years. The material is based on a course taught at the Beijing Institute of Mathematical Sciences and Applications (BIMSA).

The book is organized into three main parts:

- **Part I: Generative Models** covers the mathematical foundations of modern generative modeling, including diffusion models, flow matching, variational autoencoders, normalizing flows, and generative adversarial networks.
- **Part II: Autoregressive Models** focuses on sequential generation methods, particularly transformer-based architectures and their applications in language modeling.
- **Part III: Advanced Topics** explores world models, reinforcement learning, reasoning, agents, and applications in AI for science.

Each chapter provides rigorous mathematical derivations, intuitive explanations, and practical insights that will be valuable for researchers, students, and practitioners working in the field of artificial intelligence.

Part I

Generative Models

Chapter 1

Diffusion Models: From Mathematical Foundations to Practical Implementation

1.1 Introduction

Diffusion models have emerged as one of the most powerful and versatile approaches to generative modeling in recent years. These models, inspired by non-equilibrium thermodynamics and statistical physics, have demonstrated remarkable success in generating high-quality images, audio, and other complex data types. The fundamental idea behind diffusion models is elegant yet profound: instead of directly modeling complex data distributions, we gradually transform simple noise into the target data through a carefully designed diffusion process that can be learned and reversed.

The mathematical foundations of diffusion models rest on several key concepts from probability theory, stochastic differential equations, and statistical physics. The score function, which represents the gradient of the log-probability density, plays a central role in these models. By learning to estimate this score function, we can generate samples from complex distributions using techniques like Langevin dynamics and other sampling methods.

This chapter provides a comprehensive mathematical treatment of diffusion models, starting from the fundamental principles of sampling from probability distributions and progressing through the theoretical foundations to practical implementations. We will explore the connections between different formulations, including score matching, denoising diffusion probabilistic models (DDPM), and continuous-time formulations. The chapter concludes with an examination of recent advances that have made diffusion models more efficient and practical.

1.2 Preliminary: Generating Samples from Probability Distributions

1.2.1 The Challenge of Sampling

Given a probability distribution $p(x)$, the fundamental question in generative modeling is how to sample from this distribution. For simple distributions like uniform or Gaussian distributions, sampling is straightforward using standard random number generators. For mixture Gaussian distributions, we can first determine which component to sample from and then sample from that specific Gaussian component.

However, for general probability distributions, especially those that are only known up to a normalization constant, sampling becomes significantly more challenging. This is where so-

phisticated sampling methods become essential. The most commonly used approaches include Langevin Markov Chain Monte Carlo (Langevin MCMC) sampling and Stein Variational Gradient Descent (SVGD), among others.

1.2.2 The Role of the Score Function

A typical process for generating new samples from a modeled data distribution involves several key steps. First, we need to find or approximate the probability function $p(x)$ by modeling $p(x; \theta)$ with parameters θ . Second, we often work with unnormalized distributions of the form $p(x; \theta) = \frac{1}{C(\theta)} e^{-E(x; \theta)}$ where $C(\theta) \equiv \int_x e^{-E(x; \theta)} dx$ is the normalization constant. Finally, we generate samples using appropriate sampling methods.

The Langevin Markov Chain Monte Carlo sampling method provides a particularly elegant solution to this problem. The discrete-time Langevin dynamics can be written as:

$$x_t = x_{t-1} + \frac{\Delta t}{2} \nabla_x \log p(x_{t-1}; \theta) + \sqrt{\Delta t} \epsilon, \quad \epsilon \sim N(0, 1). \quad (1.1)$$

The crucial insight here is that the gradient term $\nabla_x \log p(x_{t-1}; \theta) = -\nabla_x E(x; \theta)$ is independent of the normalization constant $C(\theta)$. This makes the method particularly attractive for sampling from unnormalized distributions.

We define the score function as:

$$S(x; \theta) \equiv \nabla \log p(x; \theta). \quad (1.2)$$

The score function represents the direction of steepest increase in the log-probability density and plays a central role in many generative modeling approaches.

1.2.3 Continuous-Time Formulation

In the continuous-time limit as $\Delta t \rightarrow 0$, the Langevin MCMC process becomes:

$$dx = \frac{1}{2} \nabla_x \log p(x) dt + dB_t, \quad (1.3)$$

where dB_t is a Brownian motion. Given a distribution of the form $p(x) = \frac{1}{C} e^{-E}$, we have:

$$dx = -\frac{1}{2} \nabla_x E dt + dB_t. \quad (1.4)$$

This continuous formulation reveals the connection to Langevin dynamics in molecular dynamics simulations. The full Langevin dynamics equation is:

$$\ddot{x} = -\nabla_x E - \gamma \dot{x} + dB_t, \quad (1.5)$$

where γ is a friction coefficient. When the friction term $\gamma \dot{x}$ dominates the acceleration term $\ddot{x}$, we obtain the overdamped Langevin equation:

$$dx = -\frac{1}{\gamma} \nabla_x E + \frac{1}{\gamma} dB_t. \quad (1.6)$$

This overdamped limit is exactly what we use in the context of diffusion models and score-based generative modeling.

1.3 Matching Score for Training

1.3.1 The Challenge of Score Matching

The key insight in score-based generative modeling is to learn the score function using neural networks. The explicit score matching (ESM) loss is defined as:

$$J_{ESM} = \frac{1}{2} \mathbb{E}_p \left[\left\| s(x; \theta) - \frac{\partial \log p(x)}{\partial x} \right\|^2 \right], \quad (1.7)$$

where $s(x; \theta)$ is our neural network approximation of the true score function.

However, this formulation is not tractable in practice because we cannot directly compute $\frac{\partial \log p(x)}{\partial x}$ from data. This is where the theoretical developments in score matching become crucial.

1.3.2 Implicit Score Matching

Implicit Score Matching (ISM) was developed to make score matching tractable. The ISM loss is defined as:

$$J_{ISM} \equiv \mathbb{E}_p \left[\frac{1}{2} \|s(x; \theta)\|^2 + \nabla \cdot s(x; \theta) \right] = J_{ESM} - C, \quad (1.8)$$

where C is a constant independent of the parameters θ .

The key insight in the proof of this equivalence involves expanding the ESM loss:

$$J_{ESM} = \frac{1}{2} \mathbb{E}_p \left[\left\| s(x; \theta) - \frac{\partial \log p(x)}{\partial x} \right\|^2 \right] \quad (1.9)$$

$$= \frac{1}{2} \mathbb{E}_p [\|s\|^2] - \mathbb{E}_p \left[s \cdot \frac{\partial \log p}{\partial x} \right] + \frac{1}{2} \mathbb{E}_p \left[\left\| \frac{\partial \log p}{\partial x} \right\|^2 \right]. \quad (1.10)$$

The crucial step is to simplify the cross term using integration by parts:

$$-\mathbb{E}_p \left[s \cdot \frac{\partial \log p}{\partial x} \right] = - \int_x p s \cdot \nabla \log p \, dx \quad (1.11)$$

$$= - \int_x p s \cdot \frac{\nabla p}{p} \, dx = - \int_x s \cdot \nabla p \, dx \quad (1.12)$$

$$= - \int \nabla \cdot (p s) \, dx + \int p \nabla \cdot s \, dx = \mathbb{E}_p [\nabla \cdot s]. \quad (1.13)$$

Therefore, we have $J_{ISM} = J_{ESM} - C$ where $C \equiv \frac{1}{2} \mathbb{E}_p \left[\left\| \frac{\partial \log p}{\partial x} \right\|^2 \right]$ is a constant independent of θ . This makes ISM tractable since we don't need to compute the true score function.

1.3.3 Denoising Score Matching

However, the ISM loss is not very stable to optimize for two main reasons. First, the expectation is performed over the entire distribution, which can be computationally expensive. Second, the loss can become negative and decrease to a large negative value like -10^5 , making optimization challenging.

Denoising Score Matching (DSM) was developed to address these problems. The DSM loss is defined as:

$$J_{DSM} = \frac{1}{2} \mathbb{E}_{p(x, \tilde{x})} [\|s(\tilde{x}; \theta) - \nabla_{\tilde{x}} \log p(\tilde{x}|x)\|^2]. \quad (1.14)$$

We can prove that $J_{DSM} = J_{ESM} + C$, where C is a constant. The key insight is to show that the cross terms of J_{ESM} and J_{DSM} are equal:

$$\mathbb{E}_{p(\tilde{x})} [s(\tilde{x}) \cdot \nabla_{\tilde{x}} \log p(\tilde{x})] = \mathbb{E}_{p(\tilde{x}, x)} [s(\tilde{x}) \cdot \nabla_{\tilde{x}} \log p(\tilde{x}|x)]. \quad (1.15)$$

This equivalence is established by expanding the left side and using the relationship between the marginal and conditional distributions.

However, a problem arises when $\tilde{x} \rightarrow x$. We can prove that when Gaussian noise is added by $p(\tilde{x}|x) = N(\alpha x, \sigma^2)$, the DSM loss diverges to infinity as $\tilde{x} \rightarrow x$. This is because:

$$\nabla_{\tilde{x}} \log p(\tilde{x}|x) = -\frac{\tilde{x} - \alpha x}{\sigma^2}, \quad (1.16)$$

and the expectation $\mathbb{E}_{p(x, \tilde{x})}[\|\nabla_{\tilde{x}} \log p(\tilde{x}|x)\|^2]$ goes to infinity as $\sigma \rightarrow 0$.

1.4 Relations Between Different Models

1.4.1 Gaussian Noise Addition

Let us revisit the process of adding noise from x to $\tilde{x}$. The Gaussian transition kernel for adding noise is:

$$\tilde{x} = \alpha x + \sigma \epsilon, \quad (1.17)$$

which is equivalent to the Gaussian transition kernel:

$$p(\tilde{x}|x) = N(\tilde{x}; \alpha x, \sigma^2), \quad (1.18)$$

where $\epsilon \sim N(0, 1)$ and:

$$N(\tilde{x}; \alpha x, \sigma^2) = C e^{-\frac{\|\tilde{x} - \alpha x\|^2}{2\sigma^2}}. \quad (1.19)$$

We also have:

$$\epsilon = \frac{\tilde{x} - \alpha x}{\sigma}. \quad (1.20)$$

The question naturally arises: how can we recover x given $\tilde{x}$?

1.4.2 Tweedie's Formula

Tweedie's Formula from Bayesian statistics provides a crucial connection. Given random variables x, y where $y \sim N(x, \sigma^2 I)$, i.e., $p(y|x) = N(x, \sigma^2)$, the expectation of x can be given by:

$$\mathbb{E}[x|y] = y + \sigma^2 \nabla \log p(y). \quad (1.21)$$

Letting $x \leftarrow \alpha x$ and $y \leftarrow \tilde{x}$, we have:

$$s \equiv \nabla_{\tilde{x}} \log p(\tilde{x}) = -\frac{\tilde{x} - \alpha \mathbb{E}[x|\tilde{x}]}{\sigma^2} = -\frac{1}{\sigma} \mathbb{E} \left[\frac{\tilde{x} - \alpha x}{\sigma} | \tilde{x} \right] = -\frac{\mathbb{E}[\epsilon|\tilde{x}]}{\sigma}, \quad (1.22)$$

where we have used the fact that $\epsilon = \frac{\tilde{x} - \alpha x}{\sigma}$.

1.4.3 Alternative Derivation from Score Definition

We can also derive this relationship directly from the definition of the score function. Given the noise addition process $\tilde{x} = \alpha x + \sigma \epsilon$ and the conditional distribution:

$$p(\tilde{x}|x) = C e^{-\frac{\|\tilde{x} - \alpha x\|^2}{2\sigma^2}}, \quad (1.23)$$

we have:

$$s(\tilde{x}) = \nabla_{\tilde{x}} \log p(\tilde{x}) = \frac{\nabla_{\tilde{x}} p(\tilde{x})}{p(\tilde{x})} = \frac{\int_x \nabla_{\tilde{x}} p(\tilde{x}|x) p(x) dx}{p(\tilde{x})}. \quad (1.24)$$

From the conditional distribution, we have:

$$s(\tilde{x}) = \int_x \nabla_{\tilde{x}} \log p(\tilde{x}|x) p(x|\tilde{x}) dx = \mathbb{E}_x[\nabla_{\tilde{x}} \log p(\tilde{x}|x)|\tilde{x}] = \mathbb{E}_x \left[-\frac{\tilde{x} - \alpha x}{\sigma^2} | \tilde{x} \right] = -\frac{\tilde{x} - \alpha \mathbb{E}[x|\tilde{x}]}{\sigma^2}. \quad (1.25)$$

1.4.4 Theoretical Foundation: Conditional Expectation

The theoretical foundation for these relationships lies in the theory of conditional expectation. Let X be an integrable random variable. Then for each σ -algebra $\mathcal{V}$ and $Z \in \mathcal{V}$, $Z^* = \mathbb{E}(X|\mathcal{V})$ solves the least square problem:

$$\operatorname{argmin}_{Z \in \mathcal{V}} \|Z - X\|, \quad (1.26)$$

where $\|Z\| = (\int Z^2 dP)^{\frac{1}{2}}$.

This result establishes that the conditional expectation is the optimal predictor in the mean-square sense, which provides the theoretical justification for the relationships we have derived.

1.4.5 Three Equivalent Model Formulations

Based on the above analysis, we can identify three equivalent ways to model the denoising process:

Score model: The loss function is:

$$\text{Loss}_{\text{score}} = \|s_{\theta}(\tilde{x}) - \frac{\epsilon}{\sigma}\|^2. \quad (1.27)$$

X prediction model (denoising model, x_0 model): The loss function is:

$$\text{Loss}_{x_0} = \|x_{\theta}(\tilde{x}) - x\|^2, \quad (1.28)$$

with the relationship:

$$s(\tilde{x}) = -\frac{\tilde{x} - \alpha x_{\theta^*}(\tilde{x})}{\sigma^2} \quad (1.29)$$

where $x_{\theta}(\tilde{x}) \rightarrow x_{\theta^*}(\tilde{x}) \equiv \mathbb{E}[x|\tilde{x}]$.

ϵ prediction model: The loss function is:

$$\text{Loss}_{\epsilon} = \|\epsilon_{\theta}(\tilde{x}) - \epsilon\|^2, \quad (1.30)$$

with the relationship:

$$s(\tilde{x}) = -\frac{\epsilon_{\theta^*}(\tilde{x})}{\sigma}, \quad (1.31)$$

where $\epsilon_{\theta}(\tilde{x}) \rightarrow \epsilon_{\theta^*}(\tilde{x}) \equiv \mathbb{E}[\epsilon|\tilde{x}]$.

In the optimal case (θ^*), these three formulations are consistent:

$$s_{\theta^*}(\tilde{x}) = -\frac{\tilde{x} - \alpha x_{\theta^*}(\tilde{x})}{\sigma^2} = -\frac{\epsilon_{\theta^*}(\tilde{x})}{\sigma}. \quad (1.32)$$

In practice, we use the connection between these three models without necessarily reaching the optimal parameters:

$$s_{\theta}(\tilde{x}) = -\frac{\tilde{x} - \alpha x_{\theta}(\tilde{x})}{\sigma^2} = -\frac{\epsilon_{\theta}(\tilde{x})}{\sigma}, \quad (1.33)$$

and by substituting each form into the loss definitions, we can recover all the losses given by different models.

1.5 Potential Score Matching Loss

1.5.1 Key Identity: Jacobian Switch

The Gaussian transition density is:

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \alpha_t \mathbf{x}_0, \sigma_t^2 \mathbf{I}) \propto \exp\left(-\frac{\|\mathbf{x}_t - \alpha_t \mathbf{x}_0\|^2}{2\sigma_t^2}\right). \quad (1.34)$$

A basic but crucial identity is:

$$\nabla_{\mathbf{x}_t} q(\mathbf{x}_t | \mathbf{x}_0) = -\frac{1}{\alpha_t} \nabla_{\mathbf{x}_0} q(\mathbf{x}_t | \mathbf{x}_0), \quad (\alpha_t > 0). \quad (1.35)$$

The proof of this identity follows from differentiating the exponent with respect to $\mathbf{x}_t$ and $\mathbf{x}_0$:

$$\nabla_{\mathbf{x}_t} \|\mathbf{x}_t - \alpha_t \mathbf{x}_0\|^2 = 2(\mathbf{x}_t - \alpha_t \mathbf{x}_0), \quad \nabla_{\mathbf{x}_0} \|\mathbf{x}_t - \alpha_t \mathbf{x}_0\|^2 = -2\alpha_t(\mathbf{x}_t - \alpha_t \mathbf{x}_0), \quad (1.36)$$

thus giving the factor $-1/\alpha_t$.

1.5.2 Score as Conditional Force Expectation

With $p(\mathbf{x}_0) \propto e^{-E(\mathbf{x}_0)/k_B T}$ and $\mathbf{x}_t = \alpha_t \mathbf{x}_0 + \sigma_t \epsilon$, we have:

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) = \frac{1}{\alpha_t} \mathbb{E}_{\mathbf{x}_0 | \mathbf{x}_t} \left[\frac{\mathbf{F}(\mathbf{x}_0)}{k_B T} \right], \quad (1.37)$$

where $\mathbf{F}(\mathbf{x}_0) = -\nabla_{\mathbf{x}_0} E(\mathbf{x}_0)$.

The derivation follows from:

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) = \frac{\nabla_{\mathbf{x}_t} \int p(\mathbf{x}_0) q(\mathbf{x}_t | \mathbf{x}_0) d\mathbf{x}_0}{p(\mathbf{x}_t)} \quad (1.38)$$

$$= \frac{1}{p(\mathbf{x}_t)} \int p(\mathbf{x}_0) \nabla_{\mathbf{x}_t} q(\mathbf{x}_t | \mathbf{x}_0) d\mathbf{x}_0 \quad (1.39)$$

Using the Jacobian switch identity:

$$= \frac{1}{p(\mathbf{x}_t)} \int p(\mathbf{x}_0) \left(-\frac{1}{\alpha_t} \right) \nabla_{\mathbf{x}_0} q(\mathbf{x}_t | \mathbf{x}_0) d\mathbf{x}_0 \quad (1.40)$$

$$= -\frac{1}{\alpha_t} \frac{1}{p(\mathbf{x}_t)} \int q(\mathbf{x}_t | \mathbf{x}_0) \nabla_{\mathbf{x}_0} p(\mathbf{x}_0) d\mathbf{x}_0 \quad (1.41)$$

Finally:

$$= -\frac{1}{\alpha_t} \int \frac{p(\mathbf{x}_0) q(\mathbf{x}_t | \mathbf{x}_0)}{p(\mathbf{x}_t)} \nabla_{\mathbf{x}_0} \log p(\mathbf{x}_0) d\mathbf{x}_0 \quad (1.42)$$

$$= \frac{1}{\alpha_t} \mathbb{E}_{\mathbf{x}_0 | \mathbf{x}_t} \left[\frac{-\nabla_{\mathbf{x}_0} E(\mathbf{x}_0)}{k_B T} \right] = \frac{1}{\alpha_t} \mathbb{E}_{\mathbf{x}_0 | \mathbf{x}_t} \left[\frac{\mathbf{F}(\mathbf{x}_0)}{k_B T} \right] \quad (1.43)$$

This result shows that the score function can be interpreted as a conditional expectation of the force field, providing a physical interpretation of the diffusion process.

1.6 From Theory to Practice: Implementation Approaches

Now that we have established the theoretical foundations of score matching and diffusion processes, let us explore how these concepts are implemented in practice through several key approaches: Score Matching Langevin Dynamics (SMLD), Denoising Diffusion Probabilistic Models (DDPM), and continuous formulations.

1.6.1 Score Matching Langevin Dynamics (SMLD)

SMLD adds noise in the following manner:

$$p(\tilde{x}_i|x) = N(\tilde{x}_i; x, \sigma_i^2) \equiv Ce^{-\frac{\|\tilde{x}_i - x\|^2}{2\sigma_i^2}}, \quad (1.44)$$

where a geometric sequence $\sigma_1 > \sigma_2 > \dots > \sigma_T \approx 0$ is used to add noise with different levels.

In terms of random variables:

$$\tilde{x}_i = x + \sigma_i \epsilon \quad (1.45)$$

for different σ_i to get different noised random variables $\tilde{x}_i$.

The training objective J_{DSM} is:

$$J_{DSM} = \frac{1}{2} \mathbb{E}_{\sigma_i} \mathbb{E}_{p(x)} \mathbb{E}_{p(\tilde{x}_i|x)} \left[\left\| s_\theta(\tilde{x}_i, \sigma_i) + \frac{\tilde{x}_i - x}{\sigma_i^2} \right\|^2 \right]. \quad (1.46)$$

For sampling, annealed Langevin MCMC is used:

$$x_t = x_{t-1} + \frac{\Delta t}{2} S_\theta(x_{t-1}) + \sqrt{\Delta t} \epsilon, \quad \epsilon \sim N(0, 1). \quad (1.47)$$

1.6.2 Denoising Diffusion Probabilistic Models (DDPM)

DDPM was derived from the Evidence Lower Bound (ELBO), not directly from score matching. The main procedure involves:

Adding noise:

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon; \quad (1.48)$$

Transition kernel:

$$p(x_t|x_0) = N(x_t; \alpha_t x_0, \sigma_t^2), \quad (1.49)$$

where $\alpha_t = \prod_{i=1}^t \sqrt{1 - \beta_i}$ and $\sigma_t^2 = 1 - \alpha_t^2$.

Training Loss:

$$\mathbb{E}_{t \sim U[0,1]} \mathbb{E}_{p(x)} \mathbb{E}_{p(x_t|x)} [\|\epsilon_\theta(x_t, t) - \epsilon\|^2], \quad (1.50)$$

where we use the approximation $x_0 = \frac{x_t - \sigma_t \epsilon}{\alpha_t}$.

Sampling:

$$x_{t-1} = \frac{1}{\sqrt{1 - \beta_t}} (x_t + \beta_t s_\theta(x_t, t)) + \sqrt{\beta_t} \epsilon, \quad t = T, T-1, \dots, 1. \quad (1.51)$$

1.7 Continuous Formulations

1.7.1 Continuous Version of SMLD

Recall that $x_t = x_0 + \sigma_t \epsilon$. We can prove that:

$$x_t = x_{t-1} + \sqrt{\sigma_t^2 - \sigma_{t-1}^2} \epsilon. \quad (1.52)$$

The proof follows by recurrence:

$$x_t = x_{t-1} + \sqrt{\sigma_t^2 - \sigma_{t-1}^2} \epsilon \quad (1.53)$$

$$= x_{t-2} + \sqrt{\sigma_{t-1}^2 - \sigma_{t-2}^2} \epsilon + \sqrt{\sigma_t^2 - \sigma_{t-1}^2} \epsilon \quad (1.54)$$

$$= x_{t-2} + \sqrt{\sigma_t^2 - \sigma_{t-2}^2} \epsilon \quad (1.55)$$

$$= \dots = x_0 + \sigma_t \epsilon, \quad \text{where } \sigma_0 \approx 0. \quad (1.56)$$

From this, we have:

$$\Delta x_t = \sqrt{\frac{\sigma_t^2 - \sigma_{t-1}^2}{\Delta t}} \sqrt{\Delta t} \epsilon = \sqrt{\frac{\sigma_t^2 - \sigma_{t-1}^2}{\Delta t}} \Delta B_t. \quad (1.57)$$

Taking the limit as $\Delta t \rightarrow 0$:

$$dx = \sqrt{\frac{d\sigma_t^2}{dt}} dB_t. \quad (1.58)$$

1.7.2 Continuous Version of DDPM

Recall that $x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon$. Similarly:

$$x_t \approx (1 - \frac{1}{2}\beta_t) x_{t-1} + \sqrt{\beta_t} \epsilon, \quad (1.59)$$

hence:

$$\Delta x_t = -\frac{1}{2} \tilde{\beta}_t x_{t-1} \Delta t + \sqrt{\tilde{\beta}_t} \sqrt{\Delta t} \epsilon, \quad \text{where } \tilde{\beta}_t = \frac{\beta_t}{\Delta t} \quad (1.60)$$

$$= -\frac{1}{2} \tilde{\beta}_t x_{t-1} \Delta t + \sqrt{\tilde{\beta}_t} \Delta B_t. \quad (1.61)$$

Taking the limit as $\Delta t \rightarrow 0$:

$$dx = -\frac{1}{2} \tilde{\beta}_t x dt + \sqrt{\tilde{\beta}_t} dB_t. \quad (1.62)$$

1.7.3 Variable Elimination (VE) and Variable Preservation (VP)

Summarizing the forms of continuous DDPM and SMLD, we can write the adding noise process as:

$$dx = f(x, t) dt + g(t) dB_t, \quad (1.63)$$

where the corresponding distribution $p(x, t)$ satisfies the Fokker-Planck equation:

$$\frac{\partial p(x, t)}{\partial t} + \nabla \cdot (f(x, t) p(x, t)) - \frac{1}{2} g^2(t) \Delta p(x, t) = 0. \quad (1.64)$$

The training DSM objective is:

$$J_{DSM} = \mathbb{E}_{t \sim U[0,1]} \mathbb{E}_{p(x_0)} \mathbb{E}_{p(x_t|x_0)} [\|s_\theta(x_t, t) - \nabla_{x_t} \log p(x_t|x_0)\|^2]. \quad (1.65)$$

1.7.4 Transition Kernels for VE and VP

For VE, the transition kernel is:

$$P(x(t)|x(0)) = N(x(t); x(0), \sigma^2(t) - \sigma^2(0)), \quad (1.66)$$

where $\tilde{\alpha}_t \equiv 1$ and $\tilde{\sigma}_t^2 \equiv \sigma^2(t) - \sigma^2(0) \approx \sigma^2(t)$.

For VP, the transition kernel is:

$$p(x(t)|x(0)) = N\left(x(t); x(0) e^{-\frac{1}{2} \int_0^t \tilde{\beta}(s) ds}, 1 - e^{-\int_0^t \tilde{\beta}(s) ds}\right), \quad (1.67)$$

where $\tilde{\alpha}_t \equiv e^{-\frac{1}{2} \int_0^t \tilde{\beta}(s) ds}$ and $\tilde{\sigma}_t^2 \equiv 1 - e^{-\int_0^t \tilde{\beta}(s) ds}$.

1.8 Training and Inference

1.8.1 Training Recipe

The training for VE follows this process (VP is similar):

1. Randomly choose one data point $x \sim p_{data}$; 2. Randomly sample $t \sim U[0, 1]$; 3. Randomly sample white noise $\epsilon \sim N(0, 1)$; 4. Add noise: $x_t = \tilde{\alpha}_t x + \tilde{\sigma}_t \epsilon$, where the mean value $\tilde{\alpha}_t x = x(0)$ and standard deviation $\tilde{\sigma}_t = \sqrt{\sigma^2(t) - \sigma^2(0)} \approx \sigma(t)$ in VE; 5. Compute the loss by summing the following norm over x , t and ϵ :

$$\|s_\theta(x_t, t) - \epsilon/\tilde{\sigma}_t\|. \quad (1.68)$$

1.8.2 Inference: Reverse Denoising Process

The reverse denoising process is given by:

$$dx = (f(x, t) - g^2 \nabla_x \log p(x, t))dt + g(t)dB_t, \quad (1.69)$$

or:

$$dx = (f(x, t) - \frac{1}{2}g^2 \nabla_x \log p(x, t))dt, \quad (1.70)$$

or:

$$dx = (f(x, t) - \frac{3}{2}g^2 \nabla_x \log p(x, t))dt + \sqrt{2}g(t)dB_t. \quad (1.71)$$

All these forms are equivalent and obey the same Fokker-Planck equation for $p(x, t)$.

1.8.3 Inference Recipe

The inference for VE follows this process (VP is similar):

1. Randomly generate noise at time $t = 1$ as $x(1) \sim N(0, \sigma_{max}^2)$; 2. Integrate the reverse sampling equation:

$$dx = (f(x, t) - g^2(t)s_\theta(x, t))dt + g(t)dB_t \quad (1.72)$$

or:

$$dx = (f - \frac{1}{2}g^2 s_\theta)dt \quad (1.73)$$

over the time span $[0, 1]$ to get $x(0)$; 3. Corrector process: at each internal value $x(t)$, we can relax the value $x(t)$ by a corrector, which takes the Langevin MCMC process.

1.8.4 The Corrector: Langevin MCMC

The Langevin diffusion process is:

$$x_i = x_{i-1} + \frac{\Delta t}{2} \nabla_x \log p(x_{i-1}) + \sqrt{\Delta t} \epsilon, \quad \epsilon \sim N(0, 1). \quad (1.74)$$

We can prove that the distribution $\pi(x)$ of the obtained data points $\{x_i\}_{i=1}^N$ converges to $p(x)$. The continuous form is:

$$dx = \frac{1}{2} \nabla \log p(x) dt + dB_t. \quad (1.75)$$

The corresponding Fokker-Planck equation is:

$$\frac{\partial \pi(x, t)}{\partial t} + \nabla \cdot \left(\frac{1}{2} \nabla \log p(x) \pi(x, t) \right) - \frac{1}{2} \Delta \pi(x, t) = 0. \quad (1.76)$$

With sufficient time evolution, the distribution converges to a stable distribution where:

$$\frac{\partial \pi(x, t)}{\partial t} = 0, \quad t \rightarrow \infty. \quad (1.77)$$

Hence:

$$\nabla \cdot (\pi \nabla \log p - \nabla \pi) = 0, \quad \forall x. \quad (1.78)$$

Therefore:

$$\nabla \log p = \nabla \log \pi \quad (1.79)$$

given that $\pi > 0$ almost everywhere. Hence:

$$\pi^*(x) = p(x) \quad (1.80)$$

where π^* is the stable distribution of $\pi(x, t)$.

1.9 Recent Advances and Practical Considerations

1.9.1 DDPM: Denoising Diffusion Probabilistic Models

DDPM represents a foundational approach to diffusion models. The goal is to learn a generative model $p_\theta(x_0)$ by reversing a noisy diffusion process. The model introduces latents $x_{1:T}$ with the same dimensionality as data x_0 .

The reverse (sampling) chain is:

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1} | x_t), \quad p(x_T) = \mathcal{N}(0, I), \quad (1.81)$$

$$p_\theta(x_{t-1} | x_t) = \mathcal{N}(\mu_\theta(x_t, t), \Sigma_\theta(x_t, t)). \quad (1.82)$$

The forward (fixed) diffusion is:

$$q(x_{1:T} | x_0) = \prod_{t=1}^T q(x_t | x_{t-1}), \quad q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I). \quad (1.83)$$

With $\alpha_t := 1 - \beta_t$ and $\bar{\alpha}_t := \prod_{s=1}^t \alpha_s$:

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I). \quad (1.84)$$

The training objective optimizes the negative log-likelihood via a reduced-variance ELBO:

$$\begin{aligned} \mathbb{E}[-\log p_\theta(x_0)] &\leq \underbrace{\mathbb{E}[\text{KL}(q(x_T | x_0) \| p(x_T))]}_{L_T} \\ &\quad + \sum_{t=2}^T \underbrace{\mathbb{E}[\text{KL}(q(x_{t-1} | x_t, x_0) \| p_\theta(x_{t-1} | x_t))]}_{L_{t-1}} \\ &\quad - \underbrace{\mathbb{E}[\log p_\theta(x_0 | x_1)]}_{L_0}. \end{aligned} \quad (1.85)$$

The ϵ -prediction parameterization writes the forward noising as:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \quad (1.86)$$

The reverse mean is parameterized via a network $\varepsilon_\theta(x_t, t)$:

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right). \quad (1.87)$$

The simplified objective is:

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{t \sim \mathcal{U}[1, T], x_0, \epsilon} \|\epsilon - \varepsilon_\theta(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)\|_2^2. \quad (1.88)$$

1.9.2 EDM: Elucidating the Design Space

EDM separates design choices (sampling, schedules, preconditioning, training) to simplify and speed up diffusion models. The key outcomes include faster sampling (tens of NFEs) with strong FID and modular choices for ODE view, schedule $\sigma(t)$, scaling $s(t)$, time steps $\{t_i\}$, preconditioning, and loss weighting.

The probability flow ODE with schedule $\sigma(t)$ and scaling $s(t)$ is:

$$\frac{d\mathbf{x}}{dt} = \left(\frac{\dot{\sigma}}{\sigma} + \frac{\dot{s}}{s} \right) \mathbf{x} - \frac{\dot{\sigma}s}{\sigma} \nabla_{\mathbf{x}} \log p\left(\frac{\mathbf{x}}{s}; \sigma\right). \quad (1.89)$$

Using denoising score matching:

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}; \sigma) = \frac{D(\mathbf{x}; \sigma) - \mathbf{x}}{\sigma^2}, \quad (1.90)$$

so the ODE is solved numerically by evaluating a denoiser $D_{\theta}(\mathbf{x}; \sigma)$.

1.9.3 DPM-Solver: Fast ODE Solver

DPM-Solver exploits the semi-linear structure of the diffusion ODE to deliver high-quality samples in 10-20 NFE. The key idea is to compute the linear part analytically and only approximate a special, exponentially weighted integral of the network.

The forward SDE (VP-type example) is:

$$d\mathbf{x}_t = f(t)\mathbf{x}_t dt + g(t)d\mathbf{w}_t, \quad (1.91)$$

where $f(t) = \frac{d}{dt} \log \alpha_t$ and $g^2(t) = \frac{d}{dt} \sigma_t^2 - 2 \frac{d}{dt} \log \alpha_t \cdot \sigma_t^2$.

The probability flow ODE with learned noise predictor $\varepsilon_{\theta}(\mathbf{x}, t)$ is:

$$\frac{d\mathbf{x}_t}{dt} = f(t)\mathbf{x}_t + \frac{g^2(t)}{2\sigma_t} \varepsilon_{\theta}(\mathbf{x}_t, t), \quad \mathbf{x}_T \sim \mathcal{N}(0, \tilde{\sigma}^2 \mathbf{I}). \quad (1.92)$$

The exact solution via variation of constants is:

$$\mathbf{x}_t = e^{\int_t^s f(\tau) d\tau} \mathbf{x}_s + \int_t^s e^{\int_t^r f(\tau) d\tau} \frac{g^2(\tau)}{2\sigma_{\tau}} \varepsilon_{\theta}(\mathbf{x}_{\tau}, \tau) d\tau. \quad (1.93)$$

Introducing half-logSNR $\lambda_t := \log(\alpha_t/\sigma_t)$ and using change of variables:

$$\mathbf{x}_t = \frac{\alpha_t}{\alpha_s} \mathbf{x}_s - \alpha_t \int_{\lambda_s}^{\lambda_t} e^{-\lambda} \hat{\varepsilon}_{\theta}(\hat{\mathbf{x}}_{\lambda}, \lambda) d\lambda. \quad (1.94)$$

1.9.4 Consistency Models

Consistency models enable fast 1-step generation while keeping diffusion model perks (quality, zero-shot editing, compute/quality trade-off). The idea is to learn a map that sends any point on a Probability-Flow (PF) ODE trajectory back to its origin (data).

The PF ODE from SDE is:

$$\frac{dx_t}{dt} = -ts_{\phi}(x_t, t), \quad (1.95)$$

where $s_{\phi} \approx \nabla \log p_t$ under EDM-style settings.

A consistency function $f : (x_t, t) \mapsto x_{\varepsilon}$ with self-consistency $f(x_t, t) = f(x_{\varepsilon}, t')$ if on the same trajectory. A consistency model is a neural estimator $f_{\theta}(x, t)$ of f trained to enforce self-consistency.

For one-step generation:

$$x_T \sim \mathcal{N}(0, T^2 I), \quad x_{\varepsilon} = f_{\theta}(x_T, T). \quad (1.96)$$

For few-step generation, we alternate between denoising and noise injection:

$$x \leftarrow f_{\theta}(x_T, T), \text{ then for } \tau_1 > \dots > \tau_{N-1} : \hat{x}_{\tau_n} \leftarrow x + \sqrt{\tau_n^2 - \varepsilon^2} z, z \sim \mathcal{N}(0, I); x \leftarrow f_{\theta}(\hat{x}_{\tau_n}, \tau_n). \quad (1.97)$$

1.10 Open Questions and Future Directions

Several important questions remain open in the field of diffusion models:

1. **Equivalence of reverse forms:** Please prove that the forms of the reverse process are equivalent with the same marginal distribution $p(x, t)$ given the same initial condition.

2. **Score function dynamics:** Please prove that the score function $s(x, t)$ satisfies:

$$\frac{\partial s}{\partial t} = \nabla \left(-\nabla \cdot f - \langle f, s \rangle + \frac{1}{2} g^2 \|s\|^2 + \frac{1}{2} g^2 \langle \nabla, s \rangle \right). \quad (1.98)$$

3. **Practical considerations:** Why is ISM loss not commonly used like DSM loss in diffusion models nowadays? This question invites discussion about the practical trade-offs between different loss formulations.

These questions represent active areas of research that could lead to further theoretical understanding and practical improvements in diffusion models.

1.11 Conclusion

Diffusion models represent a powerful and theoretically well-founded approach to generative modeling. The mathematical foundations, rooted in stochastic differential equations, score matching, and statistical physics, provide a solid theoretical basis for understanding these models. The connections between different formulations—score matching, DDPM, and continuous-time approaches—reveal the underlying unity of these methods.

The practical implementations, from SMLD to DDPM to modern approaches like EDM and consistency models, demonstrate the evolution of the field toward more efficient and practical methods. The theoretical understanding of the relationships between different model formulations provides valuable insights for practitioners and researchers.

As the field continues to evolve, the mathematical foundations established in this chapter will remain relevant for understanding new developments and for designing improved methods. The open questions identified provide directions for future research that could lead to further theoretical insights and practical improvements.

The success of diffusion models in generating high-quality samples across diverse domains—from images to audio to scientific data—demonstrates the power of this approach. The mathematical rigor underlying these methods ensures that they will continue to be a valuable tool in the arsenal of generative modeling techniques.

Chapter 2

Flow Matching: A Unifying Perspective on Generative Modeling

2.1 Introduction

Flow matching represents a powerful and mathematically elegant approach to generative modeling that unifies various perspectives on learning to transform simple distributions into complex data distributions. Unlike score-based methods that require estimating gradients of log-probability densities, flow matching provides direct supervision through analytically derived velocity fields.

The fundamental idea behind flow matching is to learn a velocity field $\mathbf{v}(\mathbf{x}, t)$ that transports a simple base distribution $p_1(\mathbf{x})$ to the data distribution $p_0(\mathbf{x})$ through the continuity equation:

$$\partial_t p(\mathbf{x}, t) + \nabla \cdot (p(\mathbf{x}, t) \mathbf{v}(\mathbf{x}, t)) = 0. \quad (2.1)$$

This chapter provides a comprehensive mathematical treatment of flow matching, starting from the theoretical foundations and progressing through practical implementations and recent advances.

2.2 Mathematical Foundations

2.2.1 The Continuity Equation

The continuity equation is the fundamental mathematical principle underlying flow matching. Given a velocity field $\mathbf{v}(\mathbf{x}, t)$, the probability density $p(\mathbf{x}, t)$ transported by the ordinary differential equation $\dot{\mathbf{x}} = \mathbf{v}(\mathbf{x}, t)$ satisfies:

$$\partial_t p(\mathbf{x}, t) + \nabla \cdot (p(\mathbf{x}, t) \mathbf{v}(\mathbf{x}, t)) = 0. \quad (2.2)$$

This equation ensures conservation of probability mass during the flow transformation.

2.2.2 Conditional Probability Paths

Given a data distribution $p_{\text{data}}(\mathbf{x}_0)$ and base distribution $p_{\text{base}}(\mathbf{x})$, we define conditional paths $q_t(\mathbf{x}|\mathbf{x}_0)$ such that:

$$p_t(\mathbf{x}) = \int p_{\text{data}}(\mathbf{x}_0) q_t(\mathbf{x}|\mathbf{x}_0) d\mathbf{x}_0, \quad t \in [0, 1], \quad (2.3)$$

with boundary conditions $p_0 = p_{\text{data}}$ and $p_1 = p_{\text{base}}$.

The conditional velocity field is defined as:

$$\mathbf{u}_t(\mathbf{x}|\mathbf{x}_0) := \mathbb{E} \left[\frac{d}{dt} \mathbf{x}_t \middle| \mathbf{x}_t = \mathbf{x}, \mathbf{x}_0 \right], \quad (2.4)$$

which provides analytic supervision for learning the marginal velocity field $\mathbf{v}(\mathbf{x}, t)$.

2.3 Gaussian Conditional Paths

2.3.1 Mathematical Derivation

For Gaussian conditional paths of the form:

$$q_t(\mathbf{x}|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_t(\mathbf{x}_0), \sigma_t^2 \mathbf{I}), \quad (2.5)$$

where $\mathbf{x}_t = \boldsymbol{\mu}_t(\mathbf{x}_0) + \sigma_t \boldsymbol{\epsilon}$ with $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$.

Taking the time derivative:

$$\frac{d}{dt} \mathbf{x}_t = \dot{\boldsymbol{\mu}}_t(\mathbf{x}_0) + \dot{\sigma}_t \boldsymbol{\epsilon}, \quad (2.6)$$

where $\boldsymbol{\epsilon} = \frac{\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_0)}{\sigma_t}$.

The conditional expectation gives us the target velocity:

$$\mathbf{u}_t(\mathbf{x}|\mathbf{x}_0) = \dot{\boldsymbol{\mu}}_t(\mathbf{x}_0) + \frac{\dot{\sigma}_t}{\sigma_t} (\mathbf{x} - \boldsymbol{\mu}_t(\mathbf{x}_0)). \quad (2.7)$$

2.3.2 Path Choices

Different choices of $\boldsymbol{\mu}_t(\mathbf{x}_0)$ and σ_t lead to different flow matching formulations:

1. Linear bridge (optimal transport):

- $\boldsymbol{\mu}_t(\mathbf{x}_0) = (1 - t)\mathbf{x}_0$, $\sigma_t = t\sigma_{\max}$
- Minimizes Wasserstein-2 distance
- Velocity: $\mathbf{u}_t(\mathbf{x}|\mathbf{x}_0) = \frac{\sigma_{\max}}{t} (\mathbf{x} - \mathbf{x}_0)$

2. VP-like (DDPM-inspired):

- $\boldsymbol{\mu}_t(\mathbf{x}_0) = \alpha(t)\mathbf{x}_0$, $\sigma_t = \sqrt{1 - \alpha^2(t)}$
- Preserves energy structure
- Velocity: $\mathbf{u}_t(\mathbf{x}|\mathbf{x}_0) = \dot{\alpha}(t)\mathbf{x}_0 + \frac{\dot{\sigma}_t}{\sigma_t} (\mathbf{x} - \alpha(t)\mathbf{x}_0)$

3. VE-like (SMLD-inspired):

- $\boldsymbol{\mu}_t(\mathbf{x}_0) = \mathbf{x}_0$, $\sigma_t = \sigma(t)$ increasing
- Preserves data structure
- Velocity: $\mathbf{u}_t(\mathbf{x}|\mathbf{x}_0) = \frac{\dot{\sigma}(t)}{\sigma(t)} (\mathbf{x} - \mathbf{x}_0)$

2.4 Training and Inference

2.4.1 Flow Matching Objective

The flow matching training objective is:

$$J_{\text{FM}} = \mathbb{E} [\|\mathbf{v}_\theta(\mathbf{x}_t, t) - \mathbf{u}_t(\mathbf{x}_t|\mathbf{x}_0)\|^2], \quad (2.8)$$

where the expectation is over $t \sim \mathcal{U}[0, 1]$, $\mathbf{x}_0 \sim p_{\text{data}}$, and $\mathbf{x}_t \sim q_t(\cdot|\mathbf{x}_0)$.

2.4.2 Mathematical Optimality

The population minimizer of the flow matching objective is the posterior-averaged velocity field:

$$\mathbf{v}^*(\mathbf{x}, t) = \mathbb{E} \left[\mathbf{u}_t(\mathbf{x}_t | \mathbf{x}_0) \middle| \mathbf{x}_t = \mathbf{x}, t \right]. \quad (2.9)$$

This result shows that flow matching learns the optimal velocity field that respects the continuity equation.

2.4.3 Inference

The learned velocity field $\mathbf{v}_\theta(\mathbf{x}, t)$ defines a deterministic ODE:

$$\frac{d}{dt} \mathbf{x} = \mathbf{v}_\theta(\mathbf{x}, t), \quad t \in [0, 1] \quad (2.10)$$

The generation process involves:

1. Initialize $\mathbf{x}(1) \sim p_1$ (e.g., $\mathcal{N}(0, \mathbf{I})$)
2. Integrate ODE backwards: $\frac{d}{dt} \mathbf{x} = \mathbf{v}_\theta(\mathbf{x}, t)$, $t : 1 \rightarrow 0$
3. Use standard ODE solvers (Euler, RK4, Dormand-Prince, etc.)

2.5 Connection to Diffusion Models

2.5.1 Probability Flow ODE

For a forward SDE $d\mathbf{x} = f(\mathbf{x}, t)dt + g(t)dB_t$, the probability flow ODE is:

$$\frac{d}{dt} \mathbf{x} = f(\mathbf{x}, t) - \frac{1}{2} g^2(t) \nabla_{\mathbf{x}} \log p(\mathbf{x}, t). \quad (2.11)$$

When flow matching uses Gaussian paths that match the SDE's marginal distributions p_t , the learned velocity field $\mathbf{v}_\theta$ approximates the probability flow drift:

$$\mathbf{v}_\theta(\mathbf{x}, t) \approx f(\mathbf{x}, t) - \frac{1}{2} g^2(t) \nabla_{\mathbf{x}} \log p(\mathbf{x}, t) \quad (2.12)$$

This connection allows flow matching to achieve the same approximation as diffusion models without ever computing the score function during training.

2.6 MeanFlow: Average Velocity Field

2.6.1 Motivation

MeanFlow addresses the challenge of single-step generation while maintaining high fidelity. The key idea is to learn a ground-truth average velocity field that summarizes an entire time interval.

2.6.2 Definition

For any pair $r < t$ and point $\mathbf{z}_t$ along the flow path:

$$\bar{\mathbf{u}}(\mathbf{z}_t, r, t) := \frac{1}{t - r} \int_r^t \mathbf{v}(\mathbf{z}_\tau, \tau) d\tau. \quad (2.13)$$

2.6.3 MeanFlow Identity

The core relationship is:

$$\bar{\mathbf{u}}(\mathbf{z}_t, r, t) = \mathbf{v}(\mathbf{z}_t, t) - (t - r) \frac{d}{dt} \bar{\mathbf{u}}(\mathbf{z}_t, r, t), \quad (2.14)$$

where the total derivative expands to:

$$\frac{d}{dt} \bar{\mathbf{u}}(\mathbf{z}_t, r, t) = \mathbf{v}(\mathbf{z}_t, t) \cdot \nabla_{\mathbf{z}} \bar{\mathbf{u}}(\mathbf{z}_t, r, t) + \frac{\partial}{\partial t} \bar{\mathbf{u}}(\mathbf{z}_t, r, t). \quad (2.15)$$

2.6.4 Training and Sampling

The training objective uses the MeanFlow identity to form targets:

$$\mathcal{L}(\theta) = \mathbb{E} [\|\bar{\mathbf{u}}_{\theta}(\mathbf{z}_t, r, t) - \text{sg}(\bar{\mathbf{u}}_{\text{tgt}})\|_2^2], \quad (2.16)$$

where $\text{sg}(\cdot)$ is the stop-gradient operator.

For 1-NFE sampling:

$$\mathbf{x} = \mathbf{z}_0 = \boldsymbol{\epsilon} - \bar{\mathbf{u}}_{\theta}(\boldsymbol{\epsilon}, 0, 1) \quad (2.17)$$

2.7 Recent Advances and Applications

2.7.1 Practical Considerations

Flow matching has several advantages over traditional score-based methods:

- No score estimation required during training
- Analytic supervision through conditional paths
- Deterministic sampling with ODE integration
- Direct connection to optimal transport theory

2.7.2 Applications

Flow matching has been successfully applied to:

- High-resolution image generation
- Audio synthesis
- Molecular generation
- Scientific computing applications

2.8 Conclusion

Flow matching provides a mathematically elegant and practically effective approach to generative modeling. The key insights include:

- The continuity equation provides the theoretical foundation
- Conditional paths enable analytic supervision
- The connection to diffusion models reveals the underlying unity

- MeanFlow enables efficient single-step generation

The mathematical rigor and practical effectiveness of flow matching make it a valuable tool in the arsenal of generative modeling techniques, with ongoing research exploring its applications to increasingly complex domains.

Chapter 3

Variational Autoencoders: Theory and Applications

3.1 Introduction

Variational Autoencoders (VAEs) represent a fundamental breakthrough in generative modeling that combines the power of neural networks with principled probabilistic inference. Unlike traditional approaches that require intractable likelihood computations, VAEs provide a scalable framework for learning complex data distributions through variational inference.

The key insight of VAEs is to approximate intractable posterior distributions using neural networks, enabling end-to-end training through the reparameterization trick. This chapter provides a comprehensive mathematical treatment of VAEs, from theoretical foundations to practical implementations and recent advances.

3.2 The Challenge of Generative Modeling

3.2.1 Problem Formulation

Given a dataset $\mathcal{D} = \{x^{(i)}\}_{i=1}^N$ sampled from an unknown distribution $p^*(x)$, we want to:

- Learn a model $p_\theta(x)$ that approximates $p^*(x)$
- Generate new samples from the learned distribution
- Perform inference: $p_\theta(z|x)$ for latent variables z

3.2.2 Traditional Approaches and Limitations

Maximum Likelihood Estimation (MLE):

$$\theta^* = \arg \max_{\theta} \sum_{i=1}^N \log p_\theta(x^{(i)}) \quad (3.1)$$

Problem: Direct likelihood computation is often intractable for complex models.

Latent Variable Models:

$$p_\theta(x) = \int p_\theta(x|z)p_\theta(z)dz \quad (3.2)$$

Problem: Integration over latent space z is typically intractable.

3.3 Variational Inference Framework

3.3.1 The ELBO

Given a latent variable model $p_\theta(x, z) = p_\theta(x|z)p_\theta(z)$, we want to approximate the intractable posterior $p_\theta(z|x)$ and maximize the marginal likelihood $p_\theta(x)$.

The KL divergence can be decomposed as:

$$\text{KL}(q_\phi(z|x) \| p_\theta(z|x)) = \log p_\theta(x) - \text{ELBO}(x; \theta, \phi) \quad (3.3)$$

where the Evidence Lower Bound is:

$$\text{ELBO}(x; \theta, \phi) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - \text{KL}(q_\phi(z|x) \| p_\theta(z)) \quad (3.4)$$

Since $\text{KL} \geq 0$, we have:

$$\log p_\theta(x) \geq \text{ELBO}(x; \theta, \phi) \quad (3.5)$$

3.3.2 ELBO Interpretation

The ELBO consists of two components:

1. **Reconstruction term:** $\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)]$
 - Encourages $q_\phi(z|x)$ to encode information about x
 - Maximizes likelihood of reconstructing x from z
2. **Regularization term:** $-\text{KL}(q_\phi(z|x) \| p_\theta(z))$
 - Encourages $q_\phi(z|x)$ to be close to prior $p_\theta(z)$
 - Prevents overfitting to training data

3.4 VAE Architecture and Training

3.4.1 Architecture

Encoder (Recognition model): $q_\phi(z|x)$

- Maps data x to latent representation z
- Typically: $q_\phi(z|x) = \mathcal{N}(z; \mu_\phi(x), \sigma_\phi^2(x)I)$

Decoder (Generative model): $p_\theta(x|z)$

- Reconstructs data x from latent z
- Typically: $p_\theta(x|z) = \mathcal{N}(x; \mu_\theta(z), \sigma_\theta^2 I)$ or Bernoulli

Prior: $p_\theta(z) = \mathcal{N}(z; 0, I)$

3.4.2 The Reparameterization Trick

Problem: How to backpropagate through stochastic sampling $z \sim q_\phi(z|x)$?

Solution: Reparameterization trick

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (3.6)$$

Benefits:

- Gradient flows through deterministic path: $x \rightarrow \mu_\phi(x), \sigma_\phi(x) \rightarrow z$
- Stochasticity moved to ϵ (independent of parameters)
- Enables end-to-end training with backpropagation

3.4.3 Training Objective

For a dataset $\mathcal{D} = \{x^{(i)}\}_{i=1}^N$, the VAE loss is:

$$\mathcal{L}(\theta, \phi) = \sum_{i=1}^N \text{ELBO}(x^{(i)}; \theta, \phi) \quad (3.7)$$

Expanding the ELBO:

$$\mathcal{L}(\theta, \phi) = \sum_{i=1}^N \left[\mathbb{E}_{q_\phi(z|x^{(i)})} [\log p_\theta(x^{(i)}|z)] - \text{KL}(q_\phi(z|x^{(i)}) \| p_\theta(z)) \right] \quad (3.8)$$

3.4.4 Analytical KL Divergence

For Gaussian distributions, the KL divergence has a closed form:

$$\text{KL}(\mathcal{N}(\mu, \sigma^2) \| \mathcal{N}(0, I)) = \frac{1}{2} \sum_{j=1}^J (\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1) \quad (3.9)$$

where J is the dimension of the latent space.

Interpretation:

- μ_j^2 : Penalizes large mean values
- $\sigma_j^2 - \log \sigma_j^2 - 1$: Encourages $\sigma_j \approx 1$
- Total effect: Regularizes latent space to be close to standard Gaussian

3.5 VAE Variants and Extensions

3.5.1 Beta-VAE: Controlling Regularization

Motivation: Balance between reconstruction and regularization

β -VAE objective:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] - \beta \text{KL}(q_\phi(z|x) \| p_\theta(z)) \quad (3.10)$$

Effect of β :

- $\beta > 1$: Stronger regularization $\Rightarrow$ Better disentanglement
- $\beta < 1$: Weaker regularization $\Rightarrow$ Better reconstruction
- $\beta = 1$: Standard VAE

3.5.2 VQ-VAE: Vector Quantized VAE

Key idea: Use discrete latent representations instead of continuous

Architecture:

- Encoder: $z_e = E(x)$ (continuous)
- Quantization: $z_q = \text{Quantize}(z_e)$ (discrete)
- Decoder: $\hat{x} = D(z_q)$

Quantization process:

$$z_q = \arg \min_k \|z_e - e_k\|^2 \quad (3.11)$$

where $\{e_k\}_{k=1}^K$ are learnable codebook vectors.

Training objective:

$$\mathcal{L} = \log p(x|z_q) + \|\text{sg}(z_e) - e_k\|^2 + \beta \|z_e - \text{sg}(e_k)\|^2 \quad (3.12)$$

where $\text{sg}(\cdot)$ is stop-gradient operator.

3.5.3 WAE: Wasserstein Autoencoder

Motivation: Replace KL divergence with Wasserstein distance

WAE objective:

$$\mathcal{L} = \mathbb{E}_{p(x)} \mathbb{E}_{q_\phi(z|x)} [c(x, D(z))] + \lambda \mathcal{D}(q_\phi(z), p(z)) \quad (3.13)$$

where:

- $c(x, D(z))$: Reconstruction cost
- $\mathcal{D}(q_\phi(z), p(z))$: Distance between aggregated posterior and prior
- λ : Regularization weight

3.5.4 VAE-GAN: Combining VAE and GAN

Idea: Use GAN discriminator to improve VAE reconstruction quality

Training objective:

$$\mathcal{L} = \mathcal{L}_{\text{VAE}} + \lambda \mathcal{L}_{\text{GAN}} \quad (3.14)$$

where:

- $\mathcal{L}_{\text{VAE}}$: Standard VAE loss
- $\mathcal{L}_{\text{GAN}}$: Adversarial loss for discriminator

3.6 Advanced Topics

3.6.1 Hierarchical VAE

Motivation: Learn hierarchical latent representations

Architecture:

$$p_\theta(x, z_1, \dots, z_L) = p_\theta(x|z_1) \prod_{l=1}^{L-1} p_\theta(z_l|z_{l+1}) p_\theta(z_L) \quad (3.15)$$

Inference:

$$q_\phi(z_{1:L}|x) = \prod_{l=1}^L q_\phi(z_l|z_{<l}, x) \quad (3.16)$$

3.6.2 Conditional VAE (C-VAE)

Extension: Incorporate conditioning information

Model:

$$p_\theta(x, z|c) = p_\theta(x|z, c) p_\theta(z) \quad (3.17)$$

Inference:

$$q_\phi(z|x, c) = \mathcal{N}(z; \mu_\phi(x, c), \sigma_\phi^2(x, c)) \quad (3.18)$$

Training:

$$\mathcal{L} = \mathbb{E}_{q_\phi(z|x, c)} [\log p_\theta(x|z, c)] - \text{KL}(q_\phi(z|x, c) \| p_\theta(z)) \quad (3.19)$$

3.6.3 Importance Weighted Autoencoder (IWAE)

Motivation: Better approximation of log-likelihood

IWAE objective:

$$\mathcal{L}_K = \mathbb{E}_{z_1, \dots, z_K \sim q_\phi(z|x)} \left[\log \frac{1}{K} \sum_{k=1}^K \frac{p_\theta(x, z_k)}{q_\phi(z_k|x)} \right] \quad (3.20)$$

Key properties:

- $\mathcal{L}_K \geq \mathcal{L}_{K-1}$ (monotonic improvement)
- $\lim_{K \rightarrow \infty} \mathcal{L}_K = \log p_\theta(x)$ (exact likelihood)
- $K = 1$ recovers standard VAE

3.7 Implementation and Practical Considerations

3.7.1 Common Issues and Solutions

Posterior collapse:

- **Symptom:** $q_\phi(z|x) \approx p_\theta(z)$ (no information in latent)
- **Causes:** Strong decoder, weak encoder
- **Solutions:** KL annealing, free bits, stronger encoder

Blurry reconstructions:

- **Cause:** Gaussian decoder assumption
- **Solutions:** VAE-GAN, better decoder architectures

Mode collapse:

- **Symptom:** Limited diversity in generated samples
- **Solutions:** Better regularization, architectural improvements

3.7.2 Training Tricks

- **KL annealing:** Gradually increase KL weight
- **Free bits:** Prevent posterior collapse
- **Warm-up:** Start with reconstruction only

3.8 Recent Advances and Applications

3.8.1 Transformer-based VAEs

- VQ-VAE-2: Hierarchical discrete representations
- DALL-E: Text-to-image generation
- VQGAN: High-quality image generation

3.8.2 Improved Training

- NVAE: Normalizing flow decoder
- VDVAE: Very deep VAE
- VDM: Diffusion-based decoder

3.8.3 Applications

- Text generation (GPT-style models)
- Image synthesis and editing
- Audio generation
- Scientific modeling

3.9 Evaluation Metrics

Reconstruction quality:

- MSE, SSIM, FID (Fréchet Inception Distance)
- Perceptual losses

Generation quality:

- FID, IS (Inception Score)
- Human evaluation

Representation quality:

- Disentanglement metrics (β -VAE metric, MIG)
- Downstream task performance
- Interpolation smoothness

3.10 Open Questions and Future Directions

3.10.1 Theoretical Understanding

- Why do VAEs work despite approximation errors?
- Optimal choice of prior distributions
- Connection to other generative models

3.10.2 Technical Challenges

- Better posterior approximations
- Scalable training for large models
- Integration with other architectures

3.10.3 Applications

- Multi-modal learning
- Scientific discovery
- Efficient compression
- Robust representations

3.11 Conclusion

VAEs have made fundamental contributions to generative modeling:

Key contributions:

- Scalable variational inference for latent variable models
- End-to-end training with reparameterization trick
- Foundation for many generative models

VAE variants address:

- Disentanglement (β -VAE)
- Discrete representations (VQ-VAE)
- Sample quality (VAE-GAN, WAE)
- Hierarchical modeling (HVAE)

Impact:

- Influenced modern generative models
- Enabled practical latent variable modeling
- Foundation for many applications

The mathematical rigor and practical effectiveness of VAEs ensure their continued relevance in the rapidly evolving field of generative modeling.

Chapter 4

Normalizing Flows: Exact Likelihood Generative Modeling

4.1 Introduction

Normalizing flows represent a powerful class of generative models that provide exact likelihood computation and exact sampling capabilities. Unlike other generative approaches that rely on approximations, normalizing flows learn invertible transformations that map simple distributions to complex data distributions while maintaining tractable likelihood evaluation.

The fundamental principle underlying normalizing flows is the change of variables theorem, which allows us to transform probability densities through invertible mappings. This chapter provides a comprehensive mathematical treatment of normalizing flows, from theoretical foundations to practical implementations and recent advances.

4.2 Mathematical Foundations

4.2.1 Change of Variables Theorem

Given a bijective transformation $T : \mathbb{R}^d \rightarrow \mathbb{R}^d$ with inverse T^{-1} , and probability densities $p_X(x)$ and $p_Y(y)$, we have:

$$p_Y(y) = p_X(T^{-1}(y)) \left| \det \frac{\partial T^{-1}(y)}{\partial y} \right| \quad (4.1)$$

where $\frac{\partial T^{-1}(y)}{\partial y}$ is the Jacobian matrix of the inverse transformation.

Key insights:

- We can transform simple distributions into complex ones
- Computing determinants is computationally expensive
- We need to design transformations with tractable Jacobians

4.2.2 Proof of Change of Variables Formula

Theorem: For bijective $T : \mathbb{R}^d \rightarrow \mathbb{R}^d$ with inverse T^{-1} ,

$$p_Y(y) = p_X(T^{-1}(y)) \left| \det \frac{\partial T^{-1}(y)}{\partial y} \right| \quad (4.2)$$

Proof: Let $A \subset \mathbb{R}^d$ be any measurable set. Then:

$$\int_A p_Y(y) dy = \mathbb{P}(Y \in A) = \mathbb{P}(X \in T^{-1}(A)) \quad (4.3)$$

Using the change of variables formula for integrals:

$$\mathbb{P}(X \in T^{-1}(A)) = \int_{T^{-1}(A)} p_X(x) dx = \int_A p_X(T^{-1}(y)) \left| \det \frac{\partial T^{-1}(y)}{\partial y} \right| dy \quad (4.4)$$

Since this holds for any measurable set A , we have the desired result.

4.2.3 Normalizing Flows

A normalizing flow is a sequence of invertible transformations:

$$\mathbf{x}_0 \xrightarrow{T_1} \mathbf{x}_1 \xrightarrow{T_2} \mathbf{x}_2 \xrightarrow{T_3} \dots \xrightarrow{T_K} \mathbf{x}_K \quad (4.5)$$

The probability density transforms as:

$$\log p(\mathbf{x}_K) = \log p(\mathbf{x}_0) + \sum_{k=1}^K \log \left| \det \frac{\partial T_k}{\partial \mathbf{x}_{k-1}} \right| \quad (4.6)$$

Goal: Learn $T_1, T_2, \dots, T_K$ such that $p(\mathbf{x}_K)$ matches the data distribution.

4.3 Autoregressive Flows

4.3.1 Autoregressive Transformations

An autoregressive transformation processes variables in order:

$$y_i = \tau(x_i; \mathbf{h}_i) \quad (4.7)$$

where $\mathbf{h}_i = h_i(x_1, x_2, \dots, x_{i-1})$ is a function of previous variables.

Key properties:

- **Triangular Jacobian:** $\frac{\partial y_i}{\partial x_j} = 0$ for $j > i$
- **Efficient determinant:** $\det J = \prod_{i=1}^d \frac{\partial y_i}{\partial x_i}$
- **Easy inverse:** Can be computed sequentially

4.3.2 Masked Autoregressive Flow (MAF)

MAF uses neural networks with masking to ensure autoregressive structure:

$$y_i = x_i \odot \exp(\alpha_i) + \beta_i \quad (4.8)$$

where α_i, β_i are outputs of a masked neural network.

Masking strategy:

- **Input mask:** Only allow connections from $x_1, \dots, x_{i-1}$ to α_i, β_i
- Ensures $\frac{\partial y_i}{\partial x_j} = 0$ for $j \geq i$
- Jacobian is lower triangular by construction

4.3.3 Inverse Autoregressive Flow (IAF)

IAF is the inverse of MAF:

$$x_i = (y_i - \beta_i) \odot \exp(-\alpha_i) \quad (4.9)$$

Key differences from MAF:

- **Fast sampling:** Can generate samples in parallel
- **Slow density evaluation:** Requires sequential computation
- **Fast training:** When used for variational inference

Training strategy: Use IAF for fast sampling, MAF for fast density evaluation.

4.3.4 TarFlow: Transformer-based Autoregressive Flow

TarFlow is a Transformer-based variant of Masked Autoregressive Flows designed for high-resolution image synthesis.

Key Features:

- **Autoregressive Transformer Blocks:** Stack of autoregressive Transformer blocks on image patches
- **Alternating Autoregression:** Alternates autoregression direction between layers
- **End-to-End Training:** Direct modeling and generation of pixels
- **State-of-the-art Performance:** First standalone normalizing flow to match diffusion model quality

Architecture:

- **Input Processing:** Images divided into patches, each patch embedded into D -dim vectors
- **Transformer Layers:** T autoregressive transformer layers with causal attention
- **Permutation Strategy:** Alternating permutations between layers for bidirectional modeling
- **Output:** Final latent representation z^T in standard Gaussian space

Forward Transformation:

$$\tilde{z}^t = \pi^t(z^t) \quad (4.10)$$

$$z_i^{t+1} = \begin{cases} \tilde{z}_i^t & i = 0 \\ (\tilde{z}_i^t - \mu_i^t(\tilde{z}_{<i}^t)) \odot \exp(-\alpha_i^t(\tilde{z}_{<i}^t)) & i > 0 \end{cases} \quad (4.11)$$

Training Loss:

$$\min_f 0.5 \|z^T\|_2^2 + \sum_{t=0}^{T-1} \sum_{i=1}^{N-1} \sum_{j=0}^{D-1} \alpha_i^t(z_{<i}^t)_j \quad (4.12)$$

4.4 Coupling Layers

4.4.1 Affine Coupling Layers

Split the input $\mathbf{x}$ into two parts: $\mathbf{x}_A$ and $\mathbf{x}_B$.

Forward transformation:

$$\mathbf{y}_A = \mathbf{x}_A \tag{4.13}$$

$$\mathbf{y}_B = \mathbf{x}_B \odot \exp(\alpha(\mathbf{x}_A)) + \beta(\mathbf{x}_A) \tag{4.14}$$

Jacobian:

$$J = \begin{pmatrix} I & 0 \\ \frac{\partial \mathbf{y}_B}{\partial \mathbf{x}_A} & \text{diag}(\exp(\alpha(\mathbf{x}_A))) \end{pmatrix} \tag{4.15}$$

Determinant: $\det J = \prod_{i \in B} \exp(\alpha_i(\mathbf{x}_A))$

4.4.2 Real NVP: Real-valued Non-volume Preserving

Real NVP uses alternating coupling layers:

1. **Checkerboard masking:** Alternate which dimensions are transformed
2. **Channel-wise masking:** For images, alternate RGB channels

Architecture:

- ResNet blocks for α and β networks
- Batch normalization between layers
- Multi-scale architecture for images

Advantages:

- **Exact likelihood:** Can compute $p(x)$ exactly
- **Exact sampling:** Can generate samples exactly
- **Stable training:** No gradient issues

4.5 Continuous Normalizing Flows

4.5.1 Neural Ordinary Differential Equations

Instead of discrete transformations, use continuous dynamics:

$$\frac{d\mathbf{x}(t)}{dt} = f_\theta(\mathbf{x}(t), t) \tag{4.16}$$

Change of variables formula:

$$\log p(\mathbf{x}(t_1)) = \log p(\mathbf{x}(t_0)) + \int_{t_0}^{t_1} \text{tr} \left(\frac{\partial f_\theta}{\partial \mathbf{x}} \right) dt \tag{4.17}$$

Advantages:

- **Flexible architecture:** Any neural network can be used
- **Adaptive computation:** Can adjust number of function evaluations
- **Memory efficient:** No need to store intermediate states

4.5.2 Proof of Neural ODE Change of Variables Formula

Theorem: For the ODE $\frac{d\mathbf{x}(t)}{dt} = f_\theta(\mathbf{x}(t), t)$, the change of variables formula is:

$$\log p(\mathbf{x}(t_1)) = \log p(\mathbf{x}(t_0)) + \int_{t_0}^{t_1} \text{tr} \left(\frac{\partial f_\theta}{\partial \mathbf{x}} \right) dt \quad (4.18)$$

Proof: The probability density $p(\mathbf{x}, t)$ satisfies the continuity equation:

$$\frac{\partial p(\mathbf{x}, t)}{\partial t} + \nabla \cdot (f_\theta(\mathbf{x}, t)p(\mathbf{x}, t)) = 0 \quad (4.19)$$

Expanding the divergence:

$$\frac{\partial p(\mathbf{x}, t)}{\partial t} + \nabla \cdot (f_\theta(\mathbf{x}, t)p(\mathbf{x}, t)) + f_\theta(\mathbf{x}, t) \cdot \nabla p(\mathbf{x}, t) = 0 \quad (4.20)$$

Taking the log-derivative and substituting the ODE:

$$\frac{d \log p(\mathbf{x}(t), t)}{dt} = -\nabla \cdot f_\theta(\mathbf{x}(t), t) \quad (4.21)$$

Integrating from t_0 to t_1 and using the trace identity $\nabla \cdot f = \text{tr} \left(\frac{\partial f}{\partial \mathbf{x}} \right)$:

$$\log p(\mathbf{x}(t_1)) = \log p(\mathbf{x}(t_0)) + \int_{t_0}^{t_1} \text{tr} \left(\frac{\partial f_\theta}{\partial \mathbf{x}} \right) dt \quad (4.22)$$

4.5.3 FFJORD: Free-form Jacobian of Reversible Dynamics

FFJORD uses the continuous change of variables formula with Hutchinson's trace estimator:

$$\text{tr}(A) = \mathbb{E}_{\mathbf{v}}[\mathbf{v}^T A \mathbf{v}] \quad (4.23)$$

where $\mathbf{v}$ is a random vector with $\mathbb{E}[\mathbf{v}\mathbf{v}^T] = I$.

Benefits:

- **Scalable:** Works for high-dimensional data
- **Flexible:** No architectural constraints
- **Exact:** No approximation errors

4.6 Conditional Normalizing Flows

4.6.1 Goal and Conditional Likelihood

Problem: Learn a high-dimensional conditional density $p_{\mathbf{Y}|\mathbf{X}}(\mathbf{y}|\mathbf{x})$ that is multi-modal and correlated across dimensions.

CNF model: Invertible, $\mathbf{x}$ -conditioned map

$$\mathbf{z} = f_\phi(\mathbf{y}; \mathbf{x}), \quad f_\phi : \mathcal{Y} \times \mathcal{X} \rightarrow \mathcal{Z} \quad (4.24)$$

with a simple conditional base $p_{\mathbf{Z}|\mathbf{X}}(\mathbf{z}|\mathbf{x})$ (e.g., diagonal Gaussian).

Change of variables (conditional):

$$p_{\mathbf{Y}|\mathbf{X}}(\mathbf{y}|\mathbf{x}) = p_{\mathbf{Z}|\mathbf{X}}(f_\phi(\mathbf{y}; \mathbf{x})|\mathbf{x}) \left| \det \frac{\partial f_\phi(\mathbf{y}; \mathbf{x})}{\partial \mathbf{y}} \right| \quad (4.25)$$

Training:

$$\max_{\phi} \mathbb{E}_{(\mathbf{x}, \mathbf{y})} \left[\log p_{\mathbf{Z}|\mathbf{X}}(f_\phi(\mathbf{y}; \mathbf{x})|\mathbf{x}) + \log \left| \det \frac{\partial f_\phi(\mathbf{y}; \mathbf{x})}{\partial \mathbf{y}} \right| \right] \quad (4.26)$$

Sampling: $\mathbf{z} \sim p_{\mathbf{Z}|\mathbf{X}}(\cdot|\mathbf{x})$, then $\mathbf{y} = f_\phi^{-1}(\mathbf{z}; \mathbf{x})$.

4.6.2 Conditioning and Architecture

Condition injection:

$$\text{Conditional prior: } p(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mu(\mathbf{x}), \text{diag}\sigma^2(\mathbf{x})) \quad (4.27)$$

$$\text{Coupling (affine): } \begin{cases} \mathbf{y}_a = \mathbf{z}_a, \\ \mathbf{y}_b = \mathbf{z}_b \odot \exp s(\mathbf{z}_a, \mathbf{x}) + t(\mathbf{z}_a, \mathbf{x}), \end{cases} \Rightarrow \log |\det J| = \sum s(\mathbf{z}_a, \mathbf{x}) \quad (4.28)$$

Architecture primitives:

- *Coupling layers* (additive/affine/rational-quadratic spline) for cheap Jacobians
- *Invertible* 1×1 *convs* or permutations to mix channels
- *Multi-scale* (squeeze & factor-out) for efficiency and expressivity

4.7 Training and Applications

4.7.1 Maximum Likelihood Training

Given data $\{\mathbf{x}^{(i)}\}_{i=1}^N$, maximize:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \log p_{\theta}(\mathbf{x}^{(i)}) \quad (4.29)$$

Gradient computation:

$$\nabla_{\theta} \mathcal{L} = \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \log p_{\theta}(\mathbf{x}^{(i)}) \quad (4.30)$$

Challenges:

- **Mode collapse:** Flow might ignore some data modes
- **Training instability:** Gradients can be large
- **Computational cost:** Jacobian computation is expensive

4.7.2 Applications

Generative modeling:

- Image generation (Real NVP, Glow)
- Audio synthesis (WaveGlow)
- Text generation (FlowSeq)

Density estimation:

- Anomaly detection
- Out-of-distribution detection
- Uncertainty quantification

Variational inference:

- Posterior approximation
- Bayesian neural networks
- Variational autoencoders

4.8 Comparison with Other Methods

4.8.1 Normalizing Flows vs. Other Generative Models

Method	Exact Likelihood	Fast Sampling	Fast Training	Stable
VAE	No	Yes	Yes	Yes
GAN	No	Yes	No	No
Diffusion	No	No	Yes	Yes
Flow	Yes	Yes	No	Yes

Table 4.1: Comparison of generative modeling approaches

Unique advantages of flows:

- **Exact likelihood:** Can compute $p(x)$ exactly
- **Exact sampling:** Can generate samples exactly
- **Latent space:** Natural representation learning
- **Interpretability:** Understandable transformations

4.9 Challenges and Future Directions

4.9.1 Current Challenges

Computational challenges:

- **Jacobian computation:** Expensive for high dimensions
- **Memory usage:** Need to store intermediate states
- **Training time:** Slower than GANs or VAEs

Modeling challenges:

- **Architectural constraints:** Must be invertible
- **Mode collapse:** May ignore some data modes
- **Expressiveness:** Limited by transformation family

Theoretical challenges:

- **Universal approximation:** Not guaranteed for all distributions
- **Convergence:** No global convergence guarantees
- **Generalization:** Limited theoretical understanding

4.9.2 Future Directions

Architectural improvements:

- **More flexible transformations:** Beyond coupling layers
- **Better conditioning:** Improved numerical stability
- **Efficient computation:** Faster Jacobian computation

Training improvements:

- **Better optimization:** More stable training
- **Regularization:** Prevent overfitting
- **Multi-scale training:** Progressive complexity

Applications:

- **High-dimensional data:** Images, videos, audio
- **Scientific computing:** Physics simulations
- **Robotics:** Motion planning and control

4.10 Conclusion

Normalizing flows provide a unique approach to generative modeling with several key advantages:

Key contributions:

- **Exact likelihood computation**
- **Exact sampling**
- **Flexible architectures**
- **Interpretable representations**

Key insights:

- **Change of variables:** Foundation of all flow-based methods
- **Jacobian computation:** Central computational challenge
- **Architectural design:** Balance between expressiveness and efficiency
- **Training strategies:** Maximum likelihood with proper regularization

Future work:

- **Scalability:** Handle higher-dimensional data
- **Efficiency:** Reduce computational costs
- **Theoretical understanding:** Better convergence guarantees

The mathematical rigor and practical effectiveness of normalizing flows make them a valuable tool in the arsenal of generative modeling techniques, with ongoing research exploring their applications to increasingly complex domains.

Chapter 5

Generative Adversarial Networks: Adversarial Training for Generative Modeling

5.1 Introduction

Generative Adversarial Networks (GANs) represent a revolutionary approach to generative modeling that introduced the concept of adversarial training. Unlike traditional methods that rely on likelihood maximization, GANs use a game-theoretic framework where two neural networks compete against each other: a generator that creates fake data and a discriminator that distinguishes between real and fake data.

The key insight of GANs is to frame generative modeling as a minimax game, where the generator learns to produce samples that are indistinguishable from real data, while the discriminator learns to accurately classify samples as real or fake. This chapter provides a comprehensive mathematical treatment of GANs, from theoretical foundations to practical implementations and recent advances.

5.2 Mathematical Foundations

5.2.1 The Minimax Game

The GAN objective is formulated as a minimax game:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))] \quad (5.1)$$

where:

- G is the generator network that maps noise $z \sim p_z$ to data space
- D is the discriminator network that outputs the probability that input x is real
- p_{data} is the real data distribution
- p_z is the noise distribution (typically Gaussian)

5.2.2 Optimal Discriminator

For a fixed generator G , the optimal discriminator is:

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)} \quad (5.2)$$

where p_g is the distribution induced by the generator.

Proof: The discriminator's objective is:

$$\max_D V(D, G) = \int_x p_{\text{data}}(x) \log D(x) + p_g(x) \log(1 - D(x)) dx \quad (5.3)$$

Taking the derivative with respect to $D(x)$ and setting it to zero:

$$\frac{p_{\text{data}}(x)}{D(x)} - \frac{p_g(x)}{1 - D(x)} = 0 \quad (5.4)$$

Solving for $D(x)$ gives the optimal discriminator.

5.2.3 Global Optimum

When the generator is optimal, the global minimum of the minimax game is achieved when $p_g = p_{\text{data}}$, and the value of the game is $-\log 4$.

Proof: Substituting the optimal discriminator into the objective:

$$C(G) = \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D^*(x)] + \mathbb{E}_{x \sim p_g} [\log(1 - D^*(x))] \quad (5.5)$$

When $p_g = p_{\text{data}}$, we have $D^*(x) = \frac{1}{2}$ for all x , giving:

$$C(G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log \frac{1}{2}] + \mathbb{E}_{x \sim p_g} [\log \frac{1}{2}] = -\log 4 \quad (5.6)$$

5.3 Training Dynamics and Challenges

5.3.1 Training Algorithm

The GAN training algorithm alternates between:

1. **Discriminator update:** $\theta_d \leftarrow \theta_d + \alpha_d \nabla_{\theta_d} V(D, G)$
2. **Generator update:** $\theta_g \leftarrow \theta_g - \alpha_g \nabla_{\theta_g} V(D, G)$

5.3.2 Mode Collapse

Mode collapse occurs when the generator produces a limited variety of samples, ignoring some modes of the data distribution.

Causes:

- Generator finds a mode that consistently fools the discriminator
- Discriminator becomes too strong, causing generator to focus on specific samples
- Training instability leads to convergence to suboptimal solutions

Solutions:

- Unrolled GANs: Use multiple discriminator updates
- Feature matching: Match intermediate representations
- Minibatch discrimination: Encourage diversity in generated samples

5.3.3 Training Instability

GAN training is notoriously unstable due to:

- **Non-convex optimization:** The minimax game has no global convergence guarantees
- **Vanishing gradients:** When discriminator is too good, generator gradients vanish
- **Oscillatory behavior:** Generator and discriminator can oscillate without converging

5.4 Improved GAN Architectures

5.4.1 DCGAN: Deep Convolutional GANs

DCGAN introduced architectural guidelines for stable GAN training:

Generator architecture:

- Use transposed convolutions for upsampling
- Use batch normalization in generator (except output layer)
- Use ReLU activations in generator
- Use tanh activation in output layer

Discriminator architecture:

- Use strided convolutions for downsampling
- Use batch normalization in discriminator (except input layer)
- Use LeakyReLU activations in discriminator

5.4.2 WGAN: Wasserstein GAN

WGAN addresses training instability by using the Wasserstein distance instead of JS divergence.

Wasserstein distance:

$$W(p_{\text{data}}, p_g) = \inf_{\gamma \in \Pi(p_{\text{data}}, p_g)} \mathbb{E}_{(x,y) \sim \gamma} [\|x - y\|] \quad (5.7)$$

WGAN objective:

$$\min_G \max_{D \in \mathcal{D}} \mathbb{E}_{x \sim p_{\text{data}}} [D(x)] - \mathbb{E}_{z \sim p_z} [D(G(z))] \quad (5.8)$$

where $\mathcal{D}$ is the set of 1-Lipschitz functions.

Benefits:

- More stable training
- Meaningful loss function (correlates with sample quality)
- Reduced mode collapse

5.4.3 WGAN-GP: Gradient Penalty

WGAN-GP enforces the Lipschitz constraint through gradient penalty:

Objective:

$$\mathcal{L} = \mathbb{E}_{x \sim p_{\text{data}}} [D(x)] - \mathbb{E}_{z \sim p_z} [D(G(z))] + \lambda \mathbb{E}_{\hat{x} \sim p_{\hat{x}}} [(\|\nabla_{\hat{x}} D(\hat{x})\| - 1)^2] \quad (5.9)$$

where $\hat{x} = \epsilon x + (1 - \epsilon)G(z)$ with $\epsilon \sim U[0, 1]$.

5.5 Loss Functions and Training Techniques

5.5.1 Alternative Loss Functions

Least Squares GAN (LSGAN):

$$\min_D \frac{1}{2} \mathbb{E}_{x \sim p_{\text{data}}} [(D(x) - 1)^2] + \frac{1}{2} \mathbb{E}_{z \sim p_z} [(D(G(z)))^2] \quad (5.10)$$

Hinge Loss:

$$\mathcal{L}_D = -\mathbb{E}_{x \sim p_{\text{data}}} [\min(0, -1 + D(x))] - \mathbb{E}_{z \sim p_z} [\min(0, -1 - D(G(z)))] \quad (5.11)$$

5.5.2 Training Techniques

Spectral Normalization:

- Normalizes the weight matrices to have spectral norm 1
- Enforces Lipschitz constraint on discriminator
- More stable than gradient penalty

Progressive Growing:

- Start with low-resolution images
- Gradually increase resolution during training
- Improves training stability and sample quality

Self-Attention:

- Add self-attention layers to generator and discriminator
- Captures long-range dependencies in images
- Improves sample quality

5.6 Evaluation Metrics

5.6.1 Inception Score (IS)

The Inception Score measures both quality and diversity:

$$\text{IS} = \exp(\mathbb{E}_{x \sim p_g} [D_{KL}(p(y|x} \| p(y))]) \quad (5.12)$$

where $p(y|x)$ is the conditional class distribution and $p(y)$ is the marginal class distribution.

5.6.2 Fréchet Inception Distance (FID)

FID measures the distance between real and generated distributions in feature space:

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}) \quad (5.13)$$

where (μ_r, Σ_r) and (μ_g, Σ_g) are the mean and covariance of real and generated features.

5.6.3 Precision and Recall

Precision: Fraction of generated samples that are realistic **Recall:** Fraction of real data modes that are covered by generated samples

5.7 Recent Advances

5.7.1 StyleGAN: Style-Based Generator

StyleGAN introduces a style-based generator architecture:

Key innovations:

- **Style modulation:** Each layer is modulated by a style vector
- **Noise injection:** Add stochastic variation at each layer
- **Progressive growing:** Start with low resolution and grow
- **Mapping network:** Transform latent code to style vectors

Architecture:

$$\mathbf{w} = f(\mathbf{z}), \quad \mathbf{y} = g(\mathbf{w}, \mathbf{n}) \quad (5.14)$$

where f is the mapping network, g is the synthesis network, and $\mathbf{n}$ is noise.

5.7.2 BigGAN: Large Scale GAN Training

BigGAN demonstrates that GANs can scale to large datasets:

Key techniques:

- **Large batch sizes:** Use batch sizes of 2048 or higher
- **Class conditioning:** Use class labels for conditional generation
- **Truncation trick:** Truncate latent codes for better quality
- **Orthogonal initialization:** Initialize weights orthogonally

5.7.3 Self-Attention GAN (SAGAN)

SAGAN introduces self-attention mechanisms to GANs:

Self-attention layer:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (5.15)$$

Benefits:

- Captures long-range dependencies
- Improves sample quality
- Enables generation of complex scenes

5.8 Applications

5.8.1 Image Generation

- **Unconditional generation:** Generate images from noise
- **Conditional generation:** Generate images from class labels or text
- **Style transfer:** Transfer artistic style between images
- **Super-resolution:** Generate high-resolution images from low-resolution inputs

5.8.2 Data Augmentation

- **Synthetic data:** Generate additional training data
- **Domain adaptation:** Adapt models to new domains
- **Privacy preservation:** Generate synthetic data for privacy-sensitive applications

5.8.3 Scientific Applications

- **Molecular generation:** Generate new drug molecules
- **Protein design:** Design new protein structures
- **Material discovery:** Discover new materials with desired properties

5.9 Challenges and Limitations

5.9.1 Training Challenges

- **Instability:** Training can be unstable and sensitive to hyperparameters
- **Mode collapse:** Generator may ignore some data modes
- **Evaluation:** Difficult to evaluate GAN performance objectively
- **Computational cost:** Training can be computationally expensive

5.9.2 Theoretical Limitations

- **No convergence guarantees:** No global convergence guarantees
- **Mode collapse:** Theoretical understanding of mode collapse is limited
- **Generalization:** Limited understanding of generalization properties

5.10 Future Directions

5.10.1 Theoretical Advances

- **Convergence analysis:** Better understanding of training dynamics
- **Generalization bounds:** Theoretical guarantees for GAN performance
- **Mode collapse:** Better understanding and prevention of mode collapse

5.10.2 Architectural Improvements

- **More stable training:** New architectures that are easier to train
- **Better conditioning:** Improved conditional generation
- **Efficiency:** More efficient training and inference

5.10.3 Applications

- **High-resolution generation:** Generate higher resolution images
- **Video generation:** Extend GANs to video generation
- **3D generation:** Generate 3D objects and scenes

5.11 Conclusion

Generative Adversarial Networks have revolutionized the field of generative modeling by introducing adversarial training:

Key contributions:

- **Adversarial training:** Game-theoretic framework for generative modeling
- **High-quality samples:** Generate realistic images and other data
- **Flexible conditioning:** Support for various types of conditioning
- **Active research area:** Continues to drive innovation in generative modeling

Key insights:

- **Minimax game:** Generator and discriminator compete in a zero-sum game
- **Training challenges:** Instability and mode collapse are major challenges
- **Evaluation difficulty:** Objective evaluation of GAN performance remains challenging
- **Practical impact:** GANs have enabled many practical applications

Future work:

- **Theoretical understanding:** Better convergence and generalization guarantees
- **Training stability:** More robust training procedures
- **Evaluation metrics:** Better ways to evaluate GAN performance
- **Applications:** Extending GANs to new domains and tasks

The adversarial training paradigm introduced by GANs has fundamentally changed how we approach generative modeling, with ongoing research exploring new architectures, training techniques, and applications.

Part II

Autoregressive Models

Chapter 6

Autoregressive Models: Sequential Generation and Language Modeling

6.1 Introduction

Autoregressive models represent a fundamental approach to generative modeling that generates data sequentially, one element at a time, by modeling the conditional probability of each element given the previous elements. This approach has been particularly successful in language modeling, where the goal is to predict the next word given the previous words in a sequence.

The key insight of autoregressive models is to factorize the joint probability distribution as a product of conditional distributions:

$$p(x_1, x_2, \dots, x_T) = \prod_{t=1}^T p(x_t | x_{<t}) \quad (6.1)$$

This chapter provides a comprehensive mathematical treatment of autoregressive models, from theoretical foundations to practical implementations and recent advances, with particular focus on transformer-based architectures.

6.2 Mathematical Foundations

6.2.1 Autoregressive Factorization

Given a sequence $\mathbf{x} = (x_1, x_2, \dots, x_T)$, the autoregressive factorization is:

$$p(\mathbf{x}) = \prod_{t=1}^T p(x_t | x_1, x_2, \dots, x_{t-1}) = \prod_{t=1}^T p(x_t | \mathbf{x}_{<t}) \quad (6.2)$$

where $\mathbf{x}_{<t} = (x_1, x_2, \dots, x_{t-1})$ represents the history up to time $t - 1$.

6.2.2 Maximum Likelihood Training

The training objective is to maximize the log-likelihood:

$$\mathcal{L}(\theta) = \sum_{t=1}^T \log p_{\theta}(x_t | \mathbf{x}_{<t}) \quad (6.3)$$

This can be optimized using standard gradient descent methods.

6.2.3 Sampling

To generate new sequences, we sample from the conditional distributions:

1. Sample $x_1 \sim p_\theta(x_1)$
2. For $t = 2, \dots, T$: Sample $x_t \sim p_\theta(x_t | \mathbf{x}_{<t})$

6.3 Neural Autoregressive Models

6.3.1 Neural Language Models

Neural autoregressive models use neural networks to parameterize the conditional distributions:

$$p_\theta(x_t | \mathbf{x}_{<t}) = \text{softmax}(f_\theta(\mathbf{x}_{<t}))_{x_t} \quad (6.4)$$

where f_θ is a neural network that maps the history to logits over the vocabulary.

6.3.2 Recurrent Neural Networks (RNNs)

RNNs process sequences sequentially using hidden states:

$$h_t = f_\theta(h_{t-1}, x_t) \quad (6.5)$$

$$p(x_{t+1} | \mathbf{x}_{\leq t}) = \text{softmax}(Wh_t + b) \quad (6.6)$$

Advantages:

- Can handle variable-length sequences
- Memory of previous states
- Sequential processing is natural

Disadvantages:

- Sequential processing is slow
- Vanishing/exploding gradients
- Limited long-range dependencies

6.3.3 Long Short-Term Memory (LSTM)

LSTM addresses the vanishing gradient problem through gating mechanisms:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (\text{forget gate}) \quad (6.7)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (\text{input gate}) \quad (6.8)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (\text{candidate values}) \quad (6.9)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad (\text{cell state}) \quad (6.10)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (\text{output gate}) \quad (6.11)$$

$$h_t = o_t * \tanh(C_t) \quad (\text{hidden state}) \quad (6.12)$$

6.4 Transformer Architecture

6.4.1 Self-Attention Mechanism

The core innovation of transformers is the self-attention mechanism:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (6.13)$$

where Q , K , and V are query, key, and value matrices respectively.

6.4.2 Multi-Head Attention

Multi-head attention allows the model to attend to different representation subspaces:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (6.14)$$

where:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (6.15)$$

6.4.3 Positional Encoding

Since transformers have no inherent notion of sequence order, positional encodings are added:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}}) \quad (6.16)$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}}) \quad (6.17)$$

6.4.4 Transformer Block

A transformer block consists of:

1. Multi-head self-attention
2. Residual connection and layer normalization
3. Feed-forward network
4. Another residual connection and layer normalization

6.5 GPT: Generative Pre-trained Transformer

6.5.1 Architecture

GPT uses a decoder-only transformer architecture:

- **Decoder-only:** No encoder, only decoder blocks
- **Causal attention:** Each position can only attend to previous positions
- **Autoregressive:** Generates text one token at a time

6.5.2 Training

GPT is trained using the standard autoregressive objective:

$$\mathcal{L} = \sum_{t=1}^T \log p_{\theta}(x_t | \mathbf{x}_{<t}) \quad (6.18)$$

6.5.3 Pre-training and Fine-tuning

Pre-training: Train on large unlabeled text corpora **Fine-tuning:** Adapt to specific tasks with labeled data

6.6 BERT: Bidirectional Encoder Representations

6.6.1 Masked Language Modeling

BERT uses masked language modeling (MLM) for pre-training:

- Randomly mask 15% of tokens
- Predict masked tokens from context
- Use bidirectional context (unlike GPT)

6.6.2 Next Sentence Prediction

BERT also uses next sentence prediction (NSP):

- Given two sentences, predict if second follows first
- Helps with understanding sentence relationships

6.7 Advanced Architectures

6.7.1 T5: Text-to-Text Transfer Transformer

T5 frames all tasks as text-to-text problems:

- **Unified framework:** All tasks use same input/output format
- **Encoder-decoder:** Uses both encoder and decoder
- **Transfer learning:** Pre-train on large dataset, fine-tune on specific tasks

6.7.2 PaLM: Pathways Language Model

PaLM uses the Pathways system for efficient training:

- **Model parallelism:** Distribute model across multiple devices
- **Data parallelism:** Distribute data across multiple devices
- **Pipeline parallelism:** Overlap computation and communication

6.8 Training Techniques

6.8.1 Gradient Accumulation

For large models that don't fit in memory:

- Accumulate gradients over multiple mini-batches
- Update parameters after accumulating sufficient gradients
- Effective batch size = mini-batch size $\times$ accumulation steps

6.8.2 Mixed Precision Training

Use both FP16 and FP32 precision:

- **Forward pass:** Use FP16 for speed and memory
- **Gradient computation:** Use FP16 for gradients
- **Parameter updates:** Use FP32 for stability

6.8.3 Gradient Clipping

Prevent exploding gradients:

$$\text{if } \|\mathbf{g}\| > \text{max_norm: } \mathbf{g} \leftarrow \mathbf{g} \times \frac{\text{max_norm}}{\|\mathbf{g}\|} \quad (6.19)$$

6.9 Evaluation Metrics

6.9.1 Perplexity

Perplexity measures how well the model predicts the next token:

$$\text{Perplexity} = \exp \left(-\frac{1}{T} \sum_{t=1}^T \log p_{\theta}(x_t | \mathbf{x}_{<t}) \right) \quad (6.20)$$

Lower perplexity indicates better performance.

6.9.2 BLEU Score

BLEU measures the quality of generated text by comparing n-grams:

$$\text{BLEU} = \exp \left(\sum_{n=1}^N w_n \log p_n \right) \quad (6.21)$$

where p_n is the precision of n-grams and w_n are weights.

6.9.3 ROUGE Score

ROUGE measures recall-oriented evaluation:

- **ROUGE-N:** N-gram overlap
- **ROUGE-L:** Longest common subsequence
- **ROUGE-W:** Weighted longest common subsequence

6.10 Applications

6.10.1 Language Modeling

- **Text generation:** Generate coherent and contextually appropriate text
- **Completion:** Complete partial sentences or paragraphs
- **Question answering:** Answer questions based on context

6.10.2 Machine Translation

- **Sequence-to-sequence:** Translate between languages
- **Multilingual models:** Handle multiple languages in one model
- **Zero-shot translation:** Translate between language pairs not seen during training

6.10.3 Text Summarization

- **Extractive:** Select important sentences from source
- **Abstractive:** Generate new sentences that capture main ideas
- **Multi-document:** Summarize information from multiple sources

6.10.4 Dialog Systems

- **Conversational AI:** Generate human-like responses
- **Task-oriented:** Help users accomplish specific tasks
- **Open-domain:** Engage in general conversation

6.11 Challenges and Limitations

6.11.1 Computational Challenges

- **Quadratic complexity:** Self-attention scales as $O(n^2)$ with sequence length
- **Memory requirements:** Large models require significant memory
- **Training time:** Long training times for large models

6.11.2 Theoretical Limitations

- **Context length:** Limited by attention mechanism
- **Positional encoding:** May not generalize to longer sequences
- **Inductive bias:** Less inductive bias than RNNs

6.11.3 Practical Challenges

- **Reproducibility:** Difficult to reproduce results due to computational requirements
- **Interpretability:** Black-box nature makes interpretation difficult
- **Bias:** Models may learn and amplify societal biases

6.12 Recent Advances

6.12.1 Efficient Attention Mechanisms

Sparse Attention:

- **Local attention:** Only attend to nearby positions
- **Strided attention:** Attend to every k-th position
- **Random attention:** Randomly sample positions to attend to

Linear Attention:

- **Performer:** Use random features to approximate attention
- **Linformer:** Low-rank approximation of attention matrix
- **Linear Transformer:** Linear complexity attention

6.12.2 Long-Range Dependencies

Transformer-XL:

- **Segment-level recurrence:** Reuse hidden states across segments
- **Relative positional encoding:** Use relative positions instead of absolute
- **Longer context:** Handle longer sequences than standard transformers

Compressive Transformer:

- **Compression:** Compress old memories to save space
- **Longer memory:** Maintain longer context than Transformer-XL
- **Efficient:** More efficient than standard attention

6.13 Future Directions

6.13.1 Architectural Improvements

- **More efficient attention:** Reduce computational complexity
- **Better positional encoding:** Handle variable-length sequences
- **Hierarchical models:** Multi-scale processing

6.13.2 Training Improvements

- **More efficient training:** Reduce computational requirements
- **Better optimization:** New optimization algorithms
- **Regularization:** Better regularization techniques

6.13.3 Applications

- **Multimodal models:** Handle text, images, and other modalities
- **Code generation:** Generate and understand code
- **Scientific applications:** Apply to scientific problems

6.14 Conclusion

Autoregressive models have revolutionized natural language processing and generative modeling:

Key contributions:

- **Sequential generation:** Generate data one element at a time
- **Transformer architecture:** Efficient parallel processing
- **Pre-training:** Learn from large unlabeled datasets
- **Transfer learning:** Adapt to specific tasks through fine-tuning

Key insights:

- **Autoregressive factorization:** Factorize joint probability as product of conditionals
- **Attention mechanism:** Capture long-range dependencies efficiently
- **Pre-training importance:** Large-scale pre-training is crucial for performance
- **Scalability:** Performance improves with model size and data

Future work:

- **Efficiency:** More efficient architectures and training methods
- **Longer context:** Handle longer sequences and documents
- **Multimodal:** Extend to multiple modalities
- **Interpretability:** Better understanding of model behavior

The autoregressive approach, combined with transformer architectures, has enabled unprecedented advances in natural language processing and continues to drive innovation in the field.

Part III

Advanced Topics

Chapter 7

Discrete Diffusion Models: Extending Diffusion to Discrete Data

7.1 Introduction

While traditional diffusion models work well with continuous data like images, many important applications involve discrete data such as text, categorical variables, or discrete tokens. Discrete diffusion models extend the diffusion framework to handle discrete data by defining appropriate forward and reverse processes on discrete spaces.

This chapter provides a comprehensive treatment of discrete diffusion models, covering theoretical foundations, practical implementations, and applications to various discrete data types.

7.2 Mathematical Foundations

7.2.1 Discrete State Spaces

For discrete data, we work with finite state spaces $\mathcal{X} = \{1, 2, \dots, K\}$ where K is the number of possible states. The forward process gradually adds noise to discrete data, while the reverse process learns to denoise and recover the original data.

7.2.2 Forward Process

The forward process is defined by a transition matrix Q_t that specifies how to add noise at time t :

$$q(x_t|x_{t-1}) = \text{Cat}(x_t; p = x_{t-1}Q_t) \quad (7.1)$$

where Cat denotes the categorical distribution.

7.2.3 Reverse Process

The reverse process learns to denoise by parameterizing the transition probabilities:

$$p_\theta(x_{t-1}|x_t) = \text{Cat}(x_{t-1}; p = \text{softmax}(\log \hat{x}_{t-1} + \log Q_t^T)) \quad (7.2)$$

where $\hat{x}_{t-1}$ is the predicted logits from the neural network.

7.3 Noise Schedules for Discrete Data

7.3.1 Uniform Noise

The simplest approach is to add uniform noise:

$$Q_t = (1 - \beta_t)I + \beta_t \frac{1}{K} \mathbf{1}\mathbf{1}^T \quad (7.3)$$

where β_t is the noise schedule and $\mathbf{1}$ is the all-ones vector.

7.3.2 Gaussian-like Noise

For discrete data that can be embedded in continuous space, we can use Gaussian noise in the embedding space:

$$q(x_t|x_0) = \int q(x_t|\mathbf{z}_t)q(\mathbf{z}_t|\mathbf{z}_0)d\mathbf{z}_t \quad (7.4)$$

where $\mathbf{z}_t$ is the continuous embedding of x_t .

7.4 Training Objectives

7.4.1 Denoising Score Matching

The training objective is to minimize the cross-entropy loss:

$$\mathcal{L} = \mathbb{E}_{t,x_0,x_t}[-\log p_\theta(x_0|x_t)] \quad (7.5)$$

where x_0 is the original data and x_t is the noisy version at time t .

7.4.2 Simplified Objective

For uniform noise, the objective simplifies to:

$$\mathcal{L} = \mathbb{E}_{t,x_0,x_t}[-\log p_\theta(x_0|x_t)] \quad (7.6)$$

where x_t is sampled from the forward process.

7.5 Applications to Text

7.5.1 Text Diffusion

For text data, we can apply discrete diffusion at the token level:

- **Token-level diffusion:** Add noise to individual tokens
- **Character-level diffusion:** Add noise to individual characters
- **Subword-level diffusion:** Add noise to subword units

7.5.2 Training Procedure

1. Sample a text sequence x_0
2. Sample a time step $t \sim \text{Uniform}(0, T)$
3. Sample noisy sequence x_t from $q(x_t|x_0)$
4. Train model to predict x_0 from x_t

7.5.3 Generation Procedure

1. Start with random noise $x_T \sim \text{Uniform}(1, K)$
2. For $t = T, T-1, \dots, 1$:
 - Predict $\hat{x}_0$ from x_t
 - Sample x_{t-1} from $p_\theta(x_{t-1}|x_t)$

7.6 Advanced Techniques

7.6.1 Learned Noise Schedules

Instead of fixed noise schedules, we can learn the transition matrices:

$$Q_t = \text{softmax}(W_t) \quad (7.7)$$

where W_t is a learnable matrix.

7.6.2 Conditional Generation

For conditional generation, we can condition on additional information:

$$p_\theta(x_{t-1}|x_t, c) = \text{Cat}(x_{t-1}; p = \text{softmax}(\log \hat{x}_{t-1} + \log Q_t^T)) \quad (7.8)$$

where c is the conditioning information.

7.6.3 Classifier-Free Guidance

Similar to continuous diffusion, we can use classifier-free guidance:

$$\hat{x}_{t-1} = (1 + w)\hat{x}_{t-1}^c - w\hat{x}_{t-1}^u \quad (7.9)$$

where w is the guidance weight, and c and u denote conditional and unconditional predictions.

7.7 Discrete Diffusion for Images

7.7.1 VQ-VAE Integration

We can combine discrete diffusion with VQ-VAE:

1. Encode images to discrete tokens using VQ-VAE
2. Apply discrete diffusion to the tokens
3. Decode the denoised tokens back to images

7.7.2 Benefits

- **Exact likelihood:** Can compute exact likelihood
- **Fast sampling:** Can use fewer steps than continuous diffusion
- **Better control:** More control over the generation process

7.8 Evaluation Metrics

7.8.1 Perplexity

For text generation, we can measure perplexity:

$$\text{Perplexity} = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log p(x_i) \right) \quad (7.10)$$

7.8.2 BLEU Score

For machine translation tasks, we can use BLEU score:

$$\text{BLEU} = \exp \left(\sum_{n=1}^N w_n \log p_n \right) \quad (7.11)$$

7.8.3 FID Score

For image generation, we can use FID score:

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}) \quad (7.12)$$

7.9 Challenges and Limitations

7.9.1 Computational Challenges

- **Memory requirements:** Need to store transition matrices
- **Training time:** Can be slower than continuous diffusion
- **Sampling speed:** May require multiple steps for good quality

7.9.2 Modeling Challenges

- **Discrete nature:** Harder to model than continuous data
- **State space size:** Large state spaces can be challenging
- **Noise design:** Choosing appropriate noise schedules is difficult

7.10 Recent Advances

7.10.1 Improved Noise Schedules

- **Learned schedules:** Learn noise schedules from data
- **Adaptive schedules:** Adapt schedules based on data complexity
- **Multi-scale schedules:** Different schedules for different scales

7.10.2 Better Training Techniques

- **Curriculum learning:** Start with easy examples
- **Data augmentation:** Use data augmentation during training
- **Regularization:** Better regularization techniques

7.11 Applications

7.11.1 Text Generation

- **Story generation:** Generate coherent stories
- **Poetry generation:** Generate creative poetry
- **Code generation:** Generate programming code

7.11.2 Image Generation

- **High-resolution images:** Generate high-quality images
- **Style transfer:** Transfer artistic styles
- **Image editing:** Edit existing images

7.11.3 Scientific Applications

- **Molecular generation:** Generate new molecules
- **Protein design:** Design new proteins
- **Material discovery:** Discover new materials

7.12 Future Directions

7.12.1 Theoretical Advances

- **Better understanding:** Theoretical analysis of discrete diffusion
- **Convergence guarantees:** Better convergence guarantees
- **Optimal schedules:** Find optimal noise schedules

7.12.2 Architectural Improvements

- **More efficient models:** Reduce computational requirements
- **Better conditioning:** Improved conditional generation
- **Multi-modal models:** Handle multiple modalities

7.12.3 Applications

- **Longer sequences:** Handle longer text sequences
- **Better quality:** Improve generation quality
- **New domains:** Apply to new domains and tasks

7.13 Conclusion

Discrete diffusion models extend the powerful diffusion framework to discrete data:

Key contributions:

- **Discrete extension:** Extend diffusion to discrete state spaces
- **Flexible framework:** Handle various types of discrete data
- **Exact likelihood:** Can compute exact likelihood
- **Better control:** More control over generation process

Key insights:

- **Transition matrices:** Define noise through transition matrices
- **Training objectives:** Use cross-entropy loss for training
- **Generation process:** Iterative denoising for generation
- **Applications:** Wide range of applications to discrete data

Future work:

- **Efficiency:** More efficient training and inference
- **Quality:** Better generation quality
- **Theoretical understanding:** Better theoretical understanding
- **Applications:** New applications and domains

Discrete diffusion models provide a powerful framework for generative modeling of discrete data, with ongoing research exploring new applications and improvements.

Chapter 8

World Models: Learning Internal Representations of Environments

8.1 Introduction

World models represent a paradigm shift in reinforcement learning and generative modeling, where the goal is to learn an internal model of the environment that can be used for planning, imagination, and decision-making. Unlike traditional approaches that learn policies directly from environment interactions, world models learn to predict future states and rewards, enabling agents to plan and act in imagined scenarios.

This chapter provides a comprehensive treatment of world models, covering theoretical foundations, practical implementations, and applications to various domains.

8.2 Mathematical Foundations

8.2.1 World Model Definition

A world model is a learned representation of the environment that can predict future states and rewards given current states and actions:

$$p(s_{t+1}, r_t | s_t, a_t, h_t) \tag{8.1}$$

where:

- s_t is the state at time t
- a_t is the action at time t
- r_t is the reward at time t
- h_t is the history up to time t

8.2.2 Components of World Models

A world model typically consists of three components:

1. **Encoder:** Maps observations to latent states

$$z_t = \text{Encoder}(o_t) \tag{8.2}$$

2. **Dynamics Model:** Predicts next latent state

$$z_{t+1} = \text{Dynamics}(z_t, a_t) \tag{8.3}$$

3. **Decoder:** Reconstructs observations from latent states

$$\hat{o}_t = \text{Decoder}(z_t) \tag{8.4}$$

8.3 Training Objectives

8.3.1 Reconstruction Loss

The encoder-decoder should accurately reconstruct observations:

$$\mathcal{L}_{\text{recon}} = \mathbb{E}[\|o_t - \hat{o}_t\|^2] \quad (8.5)$$

8.3.2 Prediction Loss

The dynamics model should accurately predict future states:

$$\mathcal{L}_{\text{pred}} = \mathbb{E}[\|z_{t+1} - \hat{z}_{t+1}\|^2] \quad (8.6)$$

8.3.3 Reward Prediction

The model should also predict rewards:

$$\mathcal{L}_{\text{reward}} = \mathbb{E}[\|r_t - \hat{r}_t\|^2] \quad (8.7)$$

8.4 Architecture Designs

8.4.1 Recurrent World Models

Use recurrent neural networks to model temporal dependencies:

$$h_t = \text{RNN}(h_{t-1}, z_t, a_t) \quad (8.8)$$

$$z_{t+1} = \text{Linear}(h_t) \quad (8.9)$$

$$\hat{o}_t = \text{Decoder}(z_t) \quad (8.10)$$

8.4.2 Transformer-based World Models

Use transformers for long-range dependencies:

$$z_{t+1} = \text{Transformer}(z_1, \dots, z_t, a_1, \dots, a_t) \quad (8.11)$$

$$\hat{o}_t = \text{Decoder}(z_t) \quad (8.12)$$

8.4.3 VAE-based World Models

Use variational autoencoders for latent representations:

$$q_\phi(z_t|o_t) = \mathcal{N}(\mu_\phi(o_t), \sigma_\phi^2(o_t)) \quad (8.13)$$

$$p_\theta(o_t|z_t) = \mathcal{N}(\mu_\theta(z_t), \sigma_\theta^2(z_t)) \quad (8.14)$$

8.5 Planning with World Models

8.5.1 Model Predictive Control (MPC)

Use the world model for planning:

1. Start with current state s_0
2. For each action sequence $(a_1, \dots, a_H)$:
 - Roll out the world model: $s_{t+1} = f(s_t, a_t)$
 - Compute cumulative reward: $R = \sum_{t=1}^H r_t$
3. Choose action sequence with highest reward

8.5.2 Tree Search

Use tree search algorithms with the world model:

- **MCTS:** Monte Carlo Tree Search
- **AlphaZero:** Combine MCTS with neural networks
- **Imagination:** Use world model for imagination

8.6 Applications

8.6.1 Atari Games

World models have been successfully applied to Atari games:

- **World Models:** Learn compact representations of game states
- **Planning:** Use world model for planning
- **Imagination:** Generate imagined game scenarios

8.6.2 Robotics

World models are particularly useful in robotics:

- **Sim-to-real:** Transfer from simulation to real world
- **Planning:** Plan robot actions using world model
- **Imagination:** Imagine consequences of actions

8.6.3 Autonomous Driving

World models can help with autonomous driving:

- **Prediction:** Predict other vehicles' behavior
- **Planning:** Plan safe driving trajectories
- **Simulation:** Generate realistic driving scenarios

8.7 Challenges and Limitations

8.7.1 Model Accuracy

- **Compounding errors:** Errors accumulate over time
- **Distribution shift:** Model may not generalize to new situations
- **Uncertainty:** Model may be overconfident in predictions

8.7.2 Computational Challenges

- **Training time:** Can be expensive to train
- **Memory requirements:** Need to store large models
- **Inference speed:** May be slow for real-time applications

8.8 Recent Advances

8.8.1 Improved Architectures

- **Transformer-based:** Use transformers for better long-range dependencies
- **Attention mechanisms:** Use attention to focus on relevant information
- **Multi-scale models:** Handle different temporal scales

8.8.2 Better Training Techniques

- **Curriculum learning:** Start with easy tasks
- **Data augmentation:** Use data augmentation during training
- **Regularization:** Better regularization techniques

8.9 Future Directions

8.9.1 Theoretical Advances

- **Better understanding:** Theoretical analysis of world models
- **Convergence guarantees:** Better convergence guarantees
- **Generalization:** Better understanding of generalization

8.9.2 Architectural Improvements

- **More efficient models:** Reduce computational requirements
- **Better representations:** Learn better latent representations
- **Multi-modal models:** Handle multiple modalities

8.9.3 Applications

- **New domains:** Apply to new domains and tasks
- **Better planning:** Improve planning algorithms
- **Real-world deployment:** Deploy in real-world applications

8.10 Conclusion

World models represent a powerful approach to learning internal representations of environments:

Key contributions:

- **Internal models:** Learn internal representations of environments
- **Planning:** Use models for planning and decision-making
- **Imagination:** Generate imagined scenarios
- **Transfer:** Transfer knowledge across tasks

Key insights:

- **Representation learning:** Learn good representations of states
- **Dynamics modeling:** Model environment dynamics
- **Planning:** Use models for planning
- **Imagination:** Use models for imagination

Future work:

- **Efficiency:** More efficient training and inference
- **Accuracy:** Better model accuracy
- **Theoretical understanding:** Better theoretical understanding
- **Applications:** New applications and domains

World models provide a powerful framework for learning internal representations of environments, with ongoing research exploring new applications and improvements.

Chapter 9

Reinforcement Learning: Learning to Act in Dynamic Environments

9.1 Introduction

Reinforcement Learning (RL) is a paradigm of machine learning where an agent learns to make decisions by interacting with an environment. Unlike supervised learning, RL does not require labeled examples of correct behavior. Instead, the agent learns through trial and error, receiving feedback in the form of rewards or penalties.

This chapter provides a comprehensive treatment of reinforcement learning, covering theoretical foundations, practical algorithms, and applications to various domains.

9.2 Mathematical Foundations

9.2.1 Markov Decision Process (MDP)

An MDP is defined by the tuple (S, A, P, R, γ) where:

- S is the state space
- A is the action space
- $P(s'|s, a)$ is the transition probability
- $R(s, a)$ is the reward function
- $\gamma \in [0, 1]$ is the discount factor

9.2.2 Policy and Value Functions

Policy: A policy $\pi(a|s)$ defines the probability of taking action a in state s .

Value Function: The value function $V^\pi(s)$ represents the expected cumulative reward starting from state s and following policy π :

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t | s_0 = s \right] \quad (9.1)$$

Q-Function: The Q-function $Q^\pi(s, a)$ represents the expected cumulative reward starting from state s , taking action a , and then following policy π :

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t | s_0 = s, a_0 = a \right] \quad (9.2)$$

9.2.3 Bellman Equations

The value function satisfies the Bellman equation:

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V^\pi(s')] \quad (9.3)$$

The Q-function satisfies:

$$Q^\pi(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) \sum_{a'} \pi(a'|s') Q^\pi(s', a') \quad (9.4)$$

9.3 Value-Based Methods

9.3.1 Value Iteration

Value iteration updates the value function iteratively:

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V_k(s')] \quad (9.5)$$

9.3.2 Policy Iteration

Policy iteration alternates between policy evaluation and policy improvement:

1. **Policy Evaluation:** Compute V^π for current policy π
2. **Policy Improvement:** Update policy to be greedy with respect to V^π

9.3.3 Q-Learning

Q-learning is a model-free algorithm that learns the Q-function:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (9.6)$$

9.3.4 Deep Q-Networks (DQN)

DQN uses neural networks to approximate the Q-function:

$$Q(s, a) \approx Q_\theta(s, a) \quad (9.7)$$

Training objective:

$$\mathcal{L}(\theta) = \mathbb{E}[(r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a))^2] \quad (9.8)$$

where θ^- is the target network.

9.4 Policy-Based Methods

9.4.1 Policy Gradient

The policy gradient theorem states:

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi \left[\sum_{t=0}^T \nabla_\theta \log \pi(a_t|s_t) A_t \right] \quad (9.9)$$

where A_t is the advantage function.

9.4.2 REINFORCE

REINFORCE is a simple policy gradient algorithm:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi(a_t | s_t) R_t \right] \quad (9.10)$$

9.4.3 Actor-Critic Methods

Actor-critic methods combine policy and value function learning:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi(a_t | s_t) A_t \right] \quad (9.11)$$

$$A_t = Q(s_t, a_t) - V(s_t) \quad (9.12)$$

9.5 Advanced Algorithms

9.5.1 Proximal Policy Optimization (PPO)

PPO is a policy gradient algorithm that uses clipping to prevent large policy updates:

$$\mathcal{L}(\theta) = \mathbb{E} [\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)] \quad (9.13)$$

$$\text{where } r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}.$$

9.5.2 Trust Region Policy Optimization (TRPO)

TRPO uses a trust region to ensure policy updates don't degrade performance:

$$\max_{\theta} \mathbb{E} \left[\frac{\pi_{\theta}(a | s)}{\pi_{\theta_{old}}(a | s)} A_t \right] \quad (9.14)$$

subject to:

$$\mathbb{E}[D_{KL}(\pi_{\theta_{old}}(\cdot | s) \| \pi_{\theta}(\cdot | s))] \leq \delta \quad (9.15)$$

9.5.3 Soft Actor-Critic (SAC)

SAC is an off-policy algorithm that maximizes entropy:

$$J(\pi) = \mathbb{E} \left[\sum_{t=0}^T \gamma^t (r_t + \alpha \mathcal{H}(\pi(\cdot | s_t))) \right] \quad (9.16)$$

9.6 Function Approximation

9.6.1 Neural Networks

Neural networks can approximate value functions and policies:

$$V(s) \approx V_{\theta}(s) \quad (9.17)$$

$$Q(s, a) \approx Q_{\theta}(s, a) \quad (9.18)$$

$$\pi(a | s) \approx \pi_{\theta}(a | s) \quad (9.19)$$

9.6.2 Experience Replay

Experience replay stores past experiences and samples them randomly:

- **Buffer:** Store (s_t, a_t, r_t, s_{t+1}) tuples
- **Sampling:** Sample random batches for training
- **Benefits:** Reduces correlation, improves sample efficiency

9.6.3 Target Networks

Target networks provide stable targets for training:

- **Main network:** Updated every step
- **Target network:** Updated less frequently
- **Benefits:** Reduces instability, improves convergence

9.7 Exploration Strategies

9.7.1 Epsilon-Greedy

ϵ -greedy exploration balances exploration and exploitation:

$$a_t = \begin{cases} \arg \max_a Q(s_t, a) & \text{with probability } 1 - \epsilon \\ \text{random action} & \text{with probability } \epsilon \end{cases} \quad (9.20)$$

9.7.2 Upper Confidence Bound (UCB)

UCB balances exploration and exploitation based on uncertainty:

$$a_t = \arg \max_a \left[Q(s_t, a) + c \sqrt{\frac{\log t}{N_t(a)}} \right] \quad (9.21)$$

9.7.3 Thompson Sampling

Thompson sampling uses Bayesian inference for exploration:

$$a_t = \arg \max_a Q(s_t, a) + \sigma(s_t, a) \cdot \epsilon \quad (9.22)$$

where $\epsilon \sim \mathcal{N}(0, 1)$.

9.8 Applications

9.8.1 Games

RL has been successfully applied to games:

- **Atari:** DQN achieved human-level performance
- **Go:** AlphaGo defeated world champion
- **Chess:** AlphaZero achieved superhuman performance

9.8.2 Robotics

RL is widely used in robotics:

- **Manipulation:** Learn to manipulate objects
- **Locomotion:** Learn to walk and run
- **Navigation:** Learn to navigate environments

9.8.3 Autonomous Systems

RL is used in autonomous systems:

- **Autonomous driving:** Learn to drive safely
- **Drone control:** Learn to fly drones
- **Resource allocation:** Learn to allocate resources

9.9 Challenges and Limitations

9.9.1 Sample Efficiency

- **High sample complexity:** RL often requires many samples
- **Exploration:** Need to explore environment effectively
- **Transfer learning:** Difficult to transfer knowledge across tasks

9.9.2 Stability

- **Training instability:** RL training can be unstable
- **Hyperparameter sensitivity:** Performance sensitive to hyperparameters
- **Convergence:** No guarantee of convergence to optimal policy

9.9.3 Safety

- **Safe exploration:** Need to explore safely
- **Robustness:** Policies may not be robust to distribution shift
- **Interpretability:** Difficult to interpret learned policies

9.10 Recent Advances

9.10.1 Model-Based RL

Model-based RL learns a model of the environment:

- **World models:** Learn internal models of environments
- **Planning:** Use models for planning
- **Sample efficiency:** Can be more sample efficient

9.10.2 Multi-Agent RL

Multi-agent RL considers multiple agents:

- **Cooperation:** Agents can cooperate
- **Competition:** Agents can compete
- **Coordination:** Agents need to coordinate

9.10.3 Hierarchical RL

Hierarchical RL uses multiple levels of abstraction:

- **High-level policies:** Abstract actions
- **Low-level policies:** Concrete actions
- **Decomposition:** Break down complex tasks

9.11 Future Directions

9.11.1 Theoretical Advances

- **Better convergence guarantees:** Improve theoretical understanding
- **Sample complexity:** Reduce sample complexity
- **Generalization:** Better understanding of generalization

9.11.2 Algorithmic Improvements

- **More efficient algorithms:** Reduce computational requirements
- **Better exploration:** Improve exploration strategies
- **Transfer learning:** Better transfer across tasks

9.11.3 Applications

- **Real-world deployment:** Deploy in real-world applications
- **New domains:** Apply to new domains and tasks
- **Safety:** Ensure safe operation

9.12 Conclusion

Reinforcement learning provides a powerful framework for learning to act in dynamic environments:

Key contributions:

- **Learning from interaction:** Learn from trial and error
- **Decision making:** Make decisions in dynamic environments
- **Optimization:** Optimize long-term rewards

- **Autonomy:** Enable autonomous systems

Key insights:

- **Value functions:** Represent expected future rewards
- **Policy optimization:** Learn to improve policies
- **Exploration:** Balance exploration and exploitation
- **Function approximation:** Use neural networks for complex environments

Future work:

- **Efficiency:** More sample-efficient algorithms
- **Safety:** Ensure safe operation
- **Theoretical understanding:** Better theoretical understanding
- **Applications:** New applications and domains

Reinforcement learning continues to drive innovation in artificial intelligence, with ongoing research exploring new algorithms, applications, and theoretical foundations.

Chapter 10

Reasoning: From Symbolic Logic to Neural Reasoning

10.1 Introduction

Reasoning represents the ability to draw logical conclusions from given premises, a fundamental aspect of human intelligence that has been a long-standing challenge in artificial intelligence. While traditional symbolic approaches to reasoning have been successful in limited domains, recent advances in neural networks have opened new possibilities for learning to reason from data.

This chapter provides a comprehensive treatment of reasoning in AI, covering both symbolic and neural approaches, their mathematical foundations, and applications to various domains.

10.2 Symbolic Reasoning

10.2.1 Propositional Logic

Propositional logic deals with propositions that can be true or false:

- **Atomic propositions:** $p, q, r, \dots$
- **Logical connectives:** $\neg, \wedge, \vee, \rightarrow, \leftrightarrow$
- **Truth tables:** Define truth values for all combinations

10.2.2 Predicate Logic

Predicate logic extends propositional logic with quantifiers:

- **Predicates:** $P(x), Q(x, y), \dots$
- **Quantifiers:** $\forall$ (for all), $\exists$ (there exists)
- **Variables:** $x, y, z, \dots$

10.2.3 Inference Rules

Modus Ponens:

$$\frac{P \rightarrow Q, P}{Q} \quad (10.1)$$

Modus Tollens:

$$\frac{P \rightarrow Q, \neg Q}{\neg P} \quad (10.2)$$

Resolution:

$$\frac{P \vee Q, \neg P \vee R}{Q \vee R} \quad (10.3)$$

10.3 Neural Reasoning

10.3.1 Neural Symbolic Reasoning

Neural symbolic reasoning combines neural networks with symbolic reasoning:

- **Neural encoding:** Encode symbolic knowledge in neural networks
- **Symbolic reasoning:** Perform logical inference
- **Neural decoding:** Convert results back to symbolic form

10.3.2 Graph Neural Networks for Reasoning

Graph Neural Networks (GNNs) can reason about relational data:

$$h_v^{(l+1)} = \text{AGGREGATE}(\{h_u^{(l)} : u \in \mathcal{N}(v)\}) \quad (10.4)$$

where $\mathcal{N}(v)$ is the neighborhood of node v .

10.3.3 Attention Mechanisms for Reasoning

Attention mechanisms can focus on relevant information for reasoning:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (10.5)$$

10.4 Mathematical Reasoning

10.4.1 Arithmetic Reasoning

Neural networks can learn to perform arithmetic operations:

- **Addition:** $a + b = c$
- **Subtraction:** $a - b = c$
- **Multiplication:** $a \times b = c$
- **Division:** $a \div b = c$

10.4.2 Algebraic Reasoning

Neural networks can solve algebraic equations:

$$ax + b = c \Rightarrow x = \frac{c - b}{a} \quad (10.6)$$

10.4.3 Geometric Reasoning

Neural networks can reason about geometric properties:

- **Shapes:** Identify and classify shapes
- **Relations:** Understand spatial relationships
- **Transformations:** Apply geometric transformations

10.5 Logical Reasoning

10.5.1 Propositional Reasoning

Neural networks can learn propositional logic:

- **Truth tables:** Learn truth values for logical connectives
- **Inference:** Perform logical inference
- **Proofs:** Generate logical proofs

10.5.2 Predicate Reasoning

Neural networks can handle predicate logic:

- **Quantifiers:** Handle universal and existential quantifiers
- **Unification:** Find substitutions for variables
- **Resolution:** Perform resolution theorem proving

10.6 Common Sense Reasoning

10.6.1 Physical Reasoning

Neural networks can reason about physical properties:

- **Gravity:** Understand effects of gravity
- **Momentum:** Understand conservation of momentum
- **Collisions:** Predict outcomes of collisions

10.6.2 Causal Reasoning

Neural networks can learn causal relationships:

- **Causality:** Identify cause-effect relationships
- **Intervention:** Predict effects of interventions
- **Counterfactuals:** Reason about counterfactual scenarios

10.7 Reasoning in Language

10.7.1 Natural Language Inference

Neural networks can perform natural language inference:

- **Entailment:** Determine if one sentence entails another
- **Contradiction:** Identify contradictory statements
- **Neutral:** Determine if statements are unrelated

10.7.2 Question Answering

Neural networks can answer questions by reasoning:

- **Reading comprehension:** Answer questions about text
- **Multi-hop reasoning:** Answer questions requiring multiple reasoning steps
- **Commonsense QA:** Answer questions requiring commonsense knowledge

10.8 Reasoning in Vision

10.8.1 Spatial Reasoning

Neural networks can reason about spatial relationships:

- **Position:** Understand object positions
- **Orientation:** Understand object orientations
- **Relations:** Understand spatial relationships between objects

10.8.2 Temporal Reasoning

Neural networks can reason about temporal sequences:

- **Order:** Understand temporal order of events
- **Duration:** Understand duration of events
- **Causality:** Understand temporal causality

10.9 Reasoning Architectures

10.9.1 Memory Networks

Memory networks use external memory for reasoning:

- **Memory:** Store facts and information
- **Attention:** Attend to relevant memories
- **Reasoning:** Perform reasoning over memories

10.9.2 Neural Turing Machines

Neural Turing Machines combine neural networks with external memory:

- **Controller:** Neural network controller
- **Memory:** External memory matrix
- **Heads:** Read and write heads

10.9.3 Transformer-based Reasoning

Transformers can perform reasoning through attention:

- **Self-attention:** Attend to different parts of input
- **Cross-attention:** Attend to different modalities
- **Multi-head attention:** Multiple attention mechanisms

10.10 Evaluation Metrics

10.10.1 Accuracy

Standard accuracy measures:

- **Exact match:** Exact match with ground truth
- **F1 score:** Harmonic mean of precision and recall
- **BLEU score:** N-gram overlap for text generation

10.10.2 Reasoning-specific Metrics

- **Logical consistency:** Check logical consistency of outputs
- **Proof correctness:** Verify correctness of proofs
- **Step-by-step accuracy:** Accuracy of individual reasoning steps

10.11 Challenges and Limitations

10.11.1 Symbolic vs. Neural

- **Symbolic:** Interpretable but limited scalability
- **Neural:** Scalable but less interpretable
- **Hybrid:** Combine benefits of both approaches

10.11.2 Generalization

- **Domain transfer:** Transfer reasoning across domains
- **Compositionality:** Compose simple reasoning steps
- **Abstraction:** Reason at different levels of abstraction

10.11.3 Interpretability

- **Black box:** Neural networks are often black boxes
- **Explanation:** Need to explain reasoning steps
- **Debugging:** Difficult to debug reasoning errors

10.12 Recent Advances

10.12.1 Large Language Models

Large language models have shown impressive reasoning capabilities:

- **GPT:** Generative pre-trained transformer
- **BERT:** Bidirectional encoder representations
- **T5:** Text-to-text transfer transformer

10.12.2 Chain-of-Thought Reasoning

Chain-of-thought reasoning generates step-by-step reasoning:

- **Step-by-step:** Generate reasoning steps
- **Verification:** Verify each reasoning step
- **Correction:** Correct errors in reasoning

10.12.3 Program Synthesis

Program synthesis generates programs to solve reasoning tasks:

- **Code generation:** Generate code for reasoning tasks
- **Verification:** Verify correctness of generated code
- **Optimization:** Optimize generated programs

10.13 Applications

10.13.1 Scientific Discovery

- **Hypothesis generation:** Generate scientific hypotheses
- **Experiment design:** Design scientific experiments
- **Data analysis:** Analyze scientific data

10.13.2 Legal Reasoning

- **Case analysis:** Analyze legal cases
- **Precedent finding:** Find relevant legal precedents
- **Argument construction:** Construct legal arguments

10.13.3 Medical Diagnosis

- **Symptom analysis:** Analyze patient symptoms
- **Differential diagnosis:** Generate differential diagnoses
- **Treatment planning:** Plan treatments

10.14 Future Directions

10.14.1 Theoretical Advances

- **Formal verification:** Verify correctness of reasoning systems
- **Theoretical guarantees:** Provide theoretical guarantees
- **Convergence:** Understand convergence properties

10.14.2 Architectural Improvements

- **Better architectures:** Design better reasoning architectures
- **Efficiency:** Improve computational efficiency
- **Scalability:** Scale to larger problems

10.14.3 Applications

- **New domains:** Apply to new domains and tasks
- **Real-world deployment:** Deploy in real-world applications
- **Human-AI collaboration:** Collaborate with humans

10.15 Conclusion

Reasoning represents a fundamental aspect of intelligence that has seen significant advances in recent years:

Key contributions:

- **Symbolic reasoning:** Traditional logical reasoning approaches
- **Neural reasoning:** Neural network-based reasoning
- **Hybrid approaches:** Combining symbolic and neural methods
- **Applications:** Wide range of applications across domains

Key insights:

- **Representation:** Good representations are crucial for reasoning
- **Attention:** Attention mechanisms enable focused reasoning
- **Memory:** External memory can enhance reasoning capabilities
- **Compositionality:** Complex reasoning can be composed from simple steps

Future work:

- **Interpretability:** Make reasoning more interpretable
- **Generalization:** Improve generalization across domains
- **Efficiency:** More efficient reasoning algorithms
- **Applications:** New applications and domains

Reasoning continues to be a central challenge in AI, with ongoing research exploring new approaches, architectures, and applications.

Chapter 11

Intelligent Agents: From Reactive to Autonomous Systems

11.1 Introduction

Intelligent agents represent autonomous entities that can perceive their environment, reason about it, and take actions to achieve their goals. The concept of agents has evolved from simple reactive systems to sophisticated autonomous entities capable of complex reasoning, learning, and decision-making.

This chapter provides a comprehensive treatment of intelligent agents, covering their theoretical foundations, architectures, learning mechanisms, and applications to various domains.

11.2 Agent Architecture

11.2.1 Basic Agent Model

An agent can be characterized by its:

- **Perception:** How it perceives the environment
- **Reasoning:** How it reasons about the environment
- **Action:** How it acts in the environment
- **Goals:** What it aims to achieve

11.2.2 Agent-Environment Interaction

The agent-environment interaction can be modeled as:

$$\text{Agent: } s_t \rightarrow a_t \tag{11.1}$$

$$\text{Environment: } (s_t, a_t) \rightarrow (s_{t+1}, r_t) \tag{11.2}$$

where s_t is the state, a_t is the action, and r_t is the reward.

11.3 Types of Agents

11.3.1 Reactive Agents

Reactive agents respond directly to environmental stimuli:

$$a_t = \pi(s_t) \tag{11.3}$$

where π is a policy function.

11.3.2 Deliberative Agents

Deliberative agents plan before acting:

1. **Goal formulation:** Determine what to achieve
2. **Planning:** Generate a plan to achieve the goal
3. **Execution:** Execute the plan

11.3.3 Hybrid Agents

Hybrid agents combine reactive and deliberative capabilities:

- **Reactive layer:** Handle immediate responses
- **Deliberative layer:** Handle complex planning
- **Coordination:** Coordinate between layers

11.4 Learning in Agents

11.4.1 Reinforcement Learning

Agents can learn through reinforcement learning:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (11.4)$$

11.4.2 Imitation Learning

Agents can learn by imitating expert behavior:

$$\pi^* = \arg \min_{\pi} \mathbb{E}_{s \sim d^{\pi}} [\ell(\pi(s), \pi^*(s))] \quad (11.5)$$

11.4.3 Transfer Learning

Agents can transfer knowledge across tasks:

- **Feature transfer:** Transfer learned features
- **Policy transfer:** Transfer learned policies
- **Model transfer:** Transfer learned models

11.5 Multi-Agent Systems

11.5.1 Cooperative Agents

Cooperative agents work together to achieve common goals:

- **Coordination:** Coordinate actions among agents
- **Communication:** Communicate information
- **Consensus:** Reach consensus on decisions

11.5.2 Competitive Agents

Competitive agents compete against each other:

- **Game theory:** Use game-theoretic approaches
- **Nash equilibrium:** Find Nash equilibria
- **Adversarial training:** Train against opponents

11.5.3 Mixed Environments

Mixed environments contain both cooperative and competitive agents:

- **Coalition formation:** Form coalitions
- **Negotiation:** Negotiate agreements
- **Conflict resolution:** Resolve conflicts

11.6 Agent Communication

11.6.1 Message Passing

Agents can communicate through message passing:

- **Synchronous:** Messages sent and received simultaneously
- **Asynchronous:** Messages sent and received at different times
- **Reliable:** Guarantee message delivery
- **Unreliable:** No guarantee of message delivery

11.6.2 Shared Memory

Agents can share information through shared memory:

- **Blackboard:** Shared blackboard for information
- **Databases:** Shared databases
- **Knowledge bases:** Shared knowledge bases

11.6.3 Protocols

Communication protocols define how agents interact:

- **Request-Response:** Request information and receive response
- **Publish-Subscribe:** Publish information and subscribe to updates
- **Auction:** Bid for resources or tasks

11.7 Agent Planning

11.7.1 Classical Planning

Classical planning assumes:

- **Deterministic:** Actions have deterministic effects
- **Fully observable:** Complete information about environment
- **Static:** Environment doesn't change during planning

11.7.2 Probabilistic Planning

Probabilistic planning handles uncertainty:

- **Stochastic actions:** Actions have probabilistic effects
- **Partial observability:** Incomplete information about environment
- **Dynamic:** Environment can change during execution

11.7.3 Hierarchical Planning

Hierarchical planning uses multiple levels of abstraction:

- **High-level plans:** Abstract plans
- **Low-level plans:** Concrete plans
- **Decomposition:** Break down complex tasks

11.8 Agent Learning

11.8.1 Online Learning

Agents can learn while acting:

- **Exploration:** Explore new actions
- **Exploitation:** Use learned knowledge
- **Balance:** Balance exploration and exploitation

11.8.2 Offline Learning

Agents can learn from historical data:

- **Batch learning:** Learn from batches of data
- **Incremental learning:** Learn incrementally from new data
- **Transfer learning:** Transfer knowledge across tasks

11.8.3 Meta-Learning

Agents can learn how to learn:

- **Learning to learn:** Learn learning algorithms
- **Few-shot learning:** Learn from few examples
- **Rapid adaptation:** Quickly adapt to new tasks

11.9 Agent Architectures

11.9.1 Belief-Desire-Intention (BDI)

BDI agents have:

- **Beliefs:** What the agent believes about the world
- **Desires:** What the agent wants to achieve
- **Intentions:** What the agent plans to do

11.9.2 Cognitive Architectures

Cognitive architectures model human-like cognition:

- **ACT-R:** Adaptive Control of Thought-Rational
- **SOAR:** State, Operator, And Result
- **CLARION:** Connectionist Learning with Adaptive Rule Induction

11.9.3 Neural Architectures

Neural architectures use neural networks:

- **Deep Q-Networks:** Deep neural networks for Q-learning
- **Policy networks:** Neural networks for policy learning
- **Value networks:** Neural networks for value function approximation

11.10 Agent Applications

11.10.1 Robotics

Agents are widely used in robotics:

- **Autonomous robots:** Robots that operate independently
- **Swarm robotics:** Multiple robots working together
- **Human-robot interaction:** Robots that interact with humans

11.10.2 Autonomous Systems

Agents are used in autonomous systems:

- **Autonomous vehicles:** Self-driving cars
- **Unmanned aerial vehicles:** Drones
- **Autonomous spacecraft:** Space exploration missions

11.10.3 Software Agents

Agents are used in software systems:

- **Web agents:** Agents that browse the web
- **Email agents:** Agents that manage email
- **Personal assistants:** Agents that help users

11.11 Agent Evaluation

11.11.1 Performance Metrics

Agents can be evaluated using various metrics:

- **Task completion:** How well agents complete tasks
- **Efficiency:** How efficiently agents use resources
- **Reliability:** How reliably agents perform
- **Safety:** How safely agents operate

11.11.2 Benchmarks

Various benchmarks exist for evaluating agents:

- **Atari:** Video game playing
- **MuJoCo:** Continuous control tasks
- **StarCraft:** Real-time strategy games
- **Dota 2:** Multi-player online battle arena

11.12 Challenges and Limitations

11.12.1 Scalability

- **State space:** Large state spaces can be challenging
- **Action space:** Large action spaces can be challenging
- **Multi-agent:** Coordination among many agents

11.12.2 Safety

- **Safe exploration:** Explore safely in dangerous environments
- **Robustness:** Handle unexpected situations
- **Verification:** Verify agent behavior

11.12.3 Ethics

- **Fairness:** Ensure fair treatment of all agents
- **Privacy:** Protect agent privacy
- **Transparency:** Make agent decisions transparent

11.13 Recent Advances

11.13.1 Large Language Models as Agents

Large language models can act as agents:

- **Text-based interaction:** Interact through text
- **Tool use:** Use external tools
- **Planning:** Plan and execute tasks

11.13.2 Multi-Modal Agents

Multi-modal agents can handle multiple modalities:

- **Vision:** Process visual information
- **Language:** Process linguistic information
- **Audio:** Process auditory information

11.13.3 Embodied Agents

Embodied agents have physical bodies:

- **Robotics:** Physical robots
- **Virtual agents:** Virtual characters
- **Simulation:** Simulated environments

11.14 Future Directions

11.14.1 Theoretical Advances

- **Formal verification:** Verify agent behavior
- **Theoretical guarantees:** Provide theoretical guarantees
- **Convergence:** Understand convergence properties

11.14.2 Architectural Improvements

- **Better architectures:** Design better agent architectures
- **Efficiency:** Improve computational efficiency
- **Scalability:** Scale to larger problems

11.14.3 Applications

- **New domains:** Apply to new domains and tasks
- **Real-world deployment:** Deploy in real-world applications
- **Human-AI collaboration:** Collaborate with humans

11.15 Conclusion

Intelligent agents represent a fundamental paradigm in artificial intelligence:

Key contributions:

- **Autonomy:** Autonomous operation in dynamic environments
- **Learning:** Learn from experience and adapt
- **Reasoning:** Reason about environment and goals
- **Interaction:** Interact with environment and other agents

Key insights:

- **Architecture:** Good agent architecture is crucial
- **Learning:** Learning enables adaptation and improvement
- **Communication:** Communication enables coordination
- **Planning:** Planning enables goal achievement

Future work:

- **Safety:** Ensure safe operation
- **Efficiency:** More efficient agents
- **Theoretical understanding:** Better theoretical understanding
- **Applications:** New applications and domains

Intelligent agents continue to be a central focus in AI research, with ongoing work exploring new architectures, learning mechanisms, and applications.

Chapter 12

AI for Science: Accelerating Scientific Discovery

12.1 Introduction

Artificial Intelligence has emerged as a powerful tool for accelerating scientific discovery across diverse domains, from physics and chemistry to biology and medicine. The integration of AI with scientific research has led to breakthroughs in drug discovery, materials science, climate modeling, and fundamental physics research.

This chapter provides a comprehensive treatment of AI for science, covering theoretical foundations, practical applications, and the transformative impact on scientific research.

12.2 Mathematical Foundations

12.2.1 Scientific Data Representation

Scientific data can be represented in various forms:

- **Time series:** $x(t) = \{x_1, x_2, \dots, x_T\}$
- **Images:** $I(x, y) \in \mathbb{R}^{H \times W \times C}$
- **Graphs:** $G = (V, E)$ where V are nodes and E are edges
- **Sequences:** $s = \{s_1, s_2, \dots, s_n\}$ (e.g., DNA sequences)

12.2.2 Physical Constraints

AI models for science must respect physical constraints:

- **Conservation laws:** Energy, momentum, mass conservation
- **Symmetries:** Rotational, translational, gauge symmetries
- **Causality:** Cause-effect relationships
- **Uncertainty:** Quantify and propagate uncertainty

12.3 Drug Discovery

12.3.1 Molecular Representation

Molecules can be represented in various ways:

- **SMILES:** String representation of molecular structure
- **Graph:** Molecular graph with atoms as nodes and bonds as edges
- **3D coordinates:** 3D spatial coordinates of atoms
- **Descriptors:** Molecular descriptors and fingerprints

12.3.2 Molecular Property Prediction

Neural networks can predict molecular properties:

$$p = f_{\theta}(\text{molecule}) \quad (12.1)$$

where p is the property of interest (e.g., solubility, toxicity).

12.3.3 Drug-Target Interaction

Predict interactions between drugs and targets:

$$\text{Interaction} = g_{\theta}(\text{drug}, \text{target}) \quad (12.2)$$

12.3.4 Molecular Generation

Generate new molecules with desired properties:

- **VAE-based:** Use variational autoencoders
- **GAN-based:** Use generative adversarial networks
- **Flow-based:** Use normalizing flows
- **Diffusion-based:** Use diffusion models

12.4 Materials Science

12.4.1 Crystal Structure Prediction

Predict crystal structures from chemical composition:

$$\text{Structure} = h_{\theta}(\text{composition}) \quad (12.3)$$

12.4.2 Property Prediction

Predict material properties:

- **Electronic properties:** Band gap, conductivity
- **Mechanical properties:** Elastic modulus, strength
- **Thermal properties:** Thermal conductivity, heat capacity
- **Magnetic properties:** Magnetization, Curie temperature

12.4.3 Materials Discovery

Discover new materials with desired properties:

- **High-throughput screening:** Screen large databases
- **Inverse design:** Design materials for specific properties
- **Multi-objective optimization:** Optimize multiple properties

12.5 Climate Science

12.5.1 Weather Prediction

Predict weather patterns using neural networks:

$$\text{Weather}_{t+1} = f_{\theta}(\text{Weather}_t, \text{Forcing}_t) \quad (12.4)$$

12.5.2 Climate Modeling

Model climate systems:

- **Atmospheric dynamics:** Model atmospheric processes
- **Ocean dynamics:** Model ocean circulation
- **Carbon cycle:** Model carbon fluxes
- **Climate change:** Predict climate change impacts

12.5.3 Extreme Weather Events

Predict and analyze extreme weather events:

- **Hurricanes:** Predict hurricane formation and intensity
- **Heat waves:** Predict heat wave occurrence
- **Droughts:** Predict drought conditions
- **Floods:** Predict flood risk

12.6 Physics

12.6.1 Particle Physics

AI applications in particle physics:

- **Event classification:** Classify particle physics events
- **Track reconstruction:** Reconstruct particle tracks
- **Jet tagging:** Identify different types of jets
- **Anomaly detection:** Detect new physics signals

12.6.2 Plasma Physics

AI applications in plasma physics:

- **Turbulence modeling:** Model plasma turbulence
- **Confinement prediction:** Predict plasma confinement
- **Control:** Control plasma parameters
- **Diagnostics:** Analyze plasma diagnostics

12.6.3 Quantum Physics

AI applications in quantum physics:

- **Quantum state tomography:** Reconstruct quantum states
- **Quantum error correction:** Correct quantum errors
- **Quantum optimization:** Solve optimization problems
- **Quantum machine learning:** Use quantum computers for ML

12.7 Biology

12.7.1 Protein Structure Prediction

Predict protein structures from sequences:

$$\text{Structure} = f_{\theta}(\text{Sequence}) \tag{12.5}$$

12.7.2 Protein Function Prediction

Predict protein functions:

- **Enzyme classification:** Classify enzymes
- **Binding sites:** Predict binding sites
- **Protein-protein interactions:** Predict interactions
- **Drug targets:** Identify drug targets

12.7.3 Genomics

AI applications in genomics:

- **Sequence analysis:** Analyze DNA/RNA sequences
- **Variant calling:** Identify genetic variants
- **Gene expression:** Predict gene expression
- **Epigenetics:** Analyze epigenetic modifications

12.8 Scientific Computing

12.8.1 Numerical Methods

AI can accelerate numerical methods:

- **Finite element methods:** Accelerate FEM simulations
- **Monte Carlo methods:** Improve MC sampling
- **Optimization:** Solve optimization problems
- **Integration:** Numerical integration

12.8.2 Surrogate Models

AI can create surrogate models for expensive simulations:

$$\text{Output} = f_{\theta}(\text{Input}) \tag{12.6}$$

where f_{θ} is much faster than the original simulation.

12.8.3 Uncertainty Quantification

Quantify uncertainty in AI predictions:

- **Bayesian neural networks:** Bayesian uncertainty
- **Ensemble methods:** Ensemble uncertainty
- **Conformal prediction:** Distribution-free uncertainty
- **Calibration:** Calibrate uncertainty estimates

12.9 Scientific Data Analysis

12.9.1 Data Preprocessing

Preprocess scientific data:

- **Noise reduction:** Remove noise from data
- **Outlier detection:** Detect and handle outliers
- **Data augmentation:** Augment limited datasets
- **Normalization:** Normalize data for analysis

12.9.2 Feature Engineering

Engineer features for scientific data:

- **Domain knowledge:** Use domain-specific features
- **Physical constraints:** Enforce physical constraints
- **Symmetries:** Respect symmetries
- **Invariants:** Use invariant features

12.9.3 Interpretability

Make AI models interpretable for scientists:

- **Feature importance:** Identify important features
- **Attention mechanisms:** Focus on relevant parts
- **Saliency maps:** Visualize model attention
- **Counterfactuals:** Generate counterfactual explanations

12.10 Challenges and Limitations

12.10.1 Data Challenges

- **Limited data:** Scientific datasets are often small
- **Noisy data:** Experimental data can be noisy
- **Imbalanced data:** Classes may be imbalanced
- **Missing data:** Some data may be missing

12.10.2 Model Challenges

- **Generalization:** Models may not generalize well
- **Interpretability:** Models may be hard to interpret
- **Uncertainty:** Quantifying uncertainty is challenging
- **Validation:** Validating models is difficult

12.10.3 Integration Challenges

- **Workflow integration:** Integrating with existing workflows
- **Expert knowledge:** Incorporating expert knowledge
- **Validation:** Validating AI predictions
- **Trust:** Building trust in AI systems

12.11 Recent Advances

12.11.1 Large Language Models for Science

Large language models can help with scientific tasks:

- **Literature review:** Summarize scientific literature
- **Hypothesis generation:** Generate scientific hypotheses
- **Code generation:** Generate scientific code
- **Data analysis:** Analyze scientific data

12.11.2 Multimodal AI

Multimodal AI can handle multiple data types:

- **Text and images:** Combine text and image data
- **Sequences and structures:** Combine sequences and structures
- **Time series and images:** Combine time series and images
- **Multiple modalities:** Handle multiple data modalities

12.11.3 Federated Learning

Federated learning enables collaborative research:

- **Privacy:** Protect sensitive data
- **Collaboration:** Enable collaboration across institutions
- **Scalability:** Scale to large datasets
- **Efficiency:** Reduce communication costs

12.12 Applications

12.12.1 COVID-19 Research

AI has been used extensively in COVID-19 research:

- **Drug repurposing:** Identify existing drugs for COVID-19
- **Vaccine design:** Design vaccines
- **Epidemiological modeling:** Model disease spread
- **Medical imaging:** Analyze medical images

12.12.2 Cancer Research

AI applications in cancer research:

- **Drug discovery:** Discover new cancer drugs
- **Personalized medicine:** Tailor treatments to patients
- **Medical imaging:** Analyze medical images
- **Biomarker discovery:** Discover new biomarkers

12.12.3 Renewable Energy

AI applications in renewable energy:

- **Solar energy:** Optimize solar panel placement
- **Wind energy:** Predict wind patterns
- **Energy storage:** Optimize energy storage
- **Grid management:** Manage power grids

12.13 Future Directions

12.13.1 Theoretical Advances

- **Physics-informed neural networks:** Incorporate physics into neural networks
- **Uncertainty quantification:** Better uncertainty quantification
- **Interpretability:** More interpretable models
- **Generalization:** Better generalization across domains

12.13.2 Methodological Improvements

- **Better architectures:** Design better architectures for science
- **Training techniques:** Improve training techniques
- **Data augmentation:** Better data augmentation methods
- **Transfer learning:** Better transfer learning across domains

12.13.3 Applications

- **New domains:** Apply to new scientific domains
- **Real-world deployment:** Deploy in real-world applications
- **Collaboration:** Enable better collaboration between AI and scientists
- **Education:** Use AI for scientific education

12.14 Conclusion

AI for science represents a transformative paradigm in scientific research:

Key contributions:

- **Accelerated discovery:** Accelerate scientific discovery
- **New insights:** Generate new scientific insights
- **Efficiency:** Improve research efficiency
- **Collaboration:** Enable new forms of collaboration

Key insights:

- **Data integration:** Integrate diverse scientific data
- **Physical constraints:** Respect physical constraints
- **Uncertainty:** Quantify and propagate uncertainty
- **Interpretability:** Make models interpretable for scientists

Future work:

- **Theoretical understanding:** Better theoretical understanding
- **Methodological advances:** New methodological advances

- **Applications:** New applications and domains
- **Collaboration:** Better collaboration between AI and scientists

AI for science continues to drive innovation in scientific research, with ongoing work exploring new applications, methodologies, and theoretical foundations.

